SEASUG 2026 - Paper 87

Informal Introduction to Hash Tables, or How to Get Along with Associative Arrays

Bartosz Jabłoński - yabwon / Warsaw University of Technology

ABSTRACT

Even though hash tables have been with us since SAS®9.1, unusual, for a "good old SAS language", the syntax of hash tables objects can be uncomfortable, hard to accept or hard to get used to for some programmers. The purpose of this article is to convince those people that advantages of the hash tables facility overcome the vague disadvantages by three orders of magnitude. The article provides a general overview and analogies between hash tables and SAS arrays, highlighting the syntax of the hash object, and presents some use cases.

Table of contents

INTODUCTION - JUST A DICTIONARY . . . 1
ARRAYS . . . 2
TEMPORARY ARRAYS . . . 3
WHAT'S THE POINT=? . . . 3
ASSOCIATIVE ARRAYS . . . 4
SIDE-BY-SIDE COMPARISON WITH ARRAYS . . . 4
THE DEVIL'S IN THE DETAILS . . . 10
DECLARATION . . . 10
LABELS . . . 11
KEYS AD DATA . . . 19
I HAVE HASH TABLE, WHAT CAN I DO WITH IT? . . . 25
HOW TO NAVIGATE OVER HASH TABLE? . . . 30
THE LIST . . . 39
EXECUTION TIME ANIMAL . . . 40
ONE OR TWO EXAMPLES . . . 43
CONCLUSION . . . 52
REFERENCES . . . 52
Appendix A - code coloring guide . . . 53
Appendix B - saslexicon query . . . 54
INDEX . . . 54

INTODUCTION - JUST A DICTIONARY

According to the Wikipedia[1]:

> *In computer science, an associative array, key-value store, map, symbol table, or* **dictionary** *is an abstract data type that stores a collection of key/value pairs, such that each possible key appears at most once in the collection. In mathematical terms, an associative array is a function with finite domain. It supports "lookup", "remove", and "insert" operations.*
> *The dictionary problem is the classic problem of designing efficient data structures that implement associative arrays. The two major solutions to the dictionary problem are* **hash tables** *and search trees. It is sometimes also possible to solve the problem using directly addressed arrays, binary search trees, or other more specialized structures.*
> *Many programming languages include associative arrays as primitive data types, while many other languages provide software libraries that support associative arrays. Content-addressable memory is a form of direct hardware-level support for associative arrays.*
> *Associative arrays have many applications including such fundamental programming patterns as memoization and the decorator pattern. The name does not come from the associative property known in mathematics. Rather, it arises from the association of values with keys. It should not be confused with associative processors.*

In other words, a hash table (or hash object[2]) in SAS is, in its simplest form, a dictionary that allows you to search or lookup data in it by providing a key paired with that data. It is as simple as that. In fact, if you

[1] See: `https://en.wikipedia.org/wiki/Associative_array`

[2] Fun fact, in early implementation, e.g. in SAS 9.1, the "associative array" was a synonym of "hash".

have any experience with MS Excel's `VLOOKUP()` function, or ever worked with Python's dictionaries or R's lists, you are already more than qualified to use hash tables in SAS in their efficient and practical way. Of course you must have noticed that I have used the "in its simplest form" term, true the devil's in details. Use cases of hash tables can be much more complicated then "just a dictionary", but also eighty to ninety percent of situations you will use hash object is the dictionary case. Having in mind this approach (i.e., the "dictionary case") the only thing you need to get used to is the syntax used in SAS to manipulate hash objects. In subsequent chapters I provide you with a comparison and analogies between SAS arrays and hash tables that hopefully make adoption of hash object easier for you.

ARRAYS

Let's start with SAS arrays. No one needs to be convinced of how convenient and practical arrays are in SAS. Suppose you have a group of variables - `Alf`, `BoB`, `Cid`, `Dex`, `Edd`, `Foo`, ..., `Zed` - and want to perform operations on them, such as aggregating their values or performing a repetitive calculation. The first step is to collect them under a shared label, such as "gadabouts", and assign each one a numerical index (1, 2, 3, ...). Instead of referring to the variables by name, you can then refer to them by their indexed positions: the first gadabout, the second gadabout, the third gadabout, and so on - rather like the numbered villains from a villains lounge in a James Bond movie.

The key used to access a variable is now a number. This is efficient because the number points directly to the data. When you write `gadabouts[3]`, SAS goes directly to that location and retrieves the value. The following code illustrates this process; "Appendix A - code coloring guide" explains color coding.

code: array of gadabouts

```
1 data _null_;
2   Alf = 123; BoB = 234; Cid = 345; Dex = 456; Edd = 567;
3
4   ARRAY gadabouts[1:5] Alf BoB Cid Dex Edd;
5
6   key = 3;
7   data = gadabouts[key];
8   put data=;
9 run;
```

The memory layout can be visualized as below[3].

Program Data Vector									
gadabouts									
[1]	[2]	[3]	[4]	[5]					
Alf	BoB	Cid	Dex	Edd	key	data	...	_N_	_ERROR_
123	234	345	456	567	3	345		1	0

the log

```
1 data=345;
```

The approach is practical not only because now you can apply a loop and iterate over the data, but also because the time required to access a particular data cell is constant! Constant means independent from data size, it does not meter if you have five, five-hundred, of five billions cells, accessing one takes always the same amount of time.

[3] Of course this is not how the memory layout is really organized under the hood, it is rather picture of how logically you can think about how the work is done. This representation is provided just for didactic purpose, to give you some perspective

You may argue that the example has no practical sense because I replaced names with "meaningless" numbers. Well in this educational example yes, but what if you consider an array of 365 cells where data are interest rates values for each day in 2026. Now the keys have a very practical meaning; 42 is the forty-second day of 2026 and it points the value reflecting state as of Wednesday, February 11th. Zip codes in Poland are 5 digits numbers (formatted 99–999). You can easily imagine an array of town names or post-office buildings addresses pointed by zip-code number. One more example from Poland, two main tax identification numbers depending on whether you are an individual or a business are used in Polish tax system: the 11-digit PESEL for private residents and the 10-digit NIP (Numer Identyfikacji Podatkowej) for businesses and active taxpayers, it is easy to imagine two arrays where the data are names and addresses of taxpayers/companies, and keys are PESEL/NIP numbers. In every case an integer key has practical business meaning and access to a data cell in every case, regardless of `365`, `99999`, or `1E10/1E11` cells, is done in constant time.

TEMPORARY ARRAYS

A "regular" SAS array is a convenience handler over variables stored Program Data Vector. A tool to have a separate block of memory available to work with is the SAS temporary array. In the code, you just add a single keyword word, `_TEMPORARY_`, and you can perform all array operations, but now you work on truly separate in-memory located object. The PDV, except variables under the array umbrella can contain additional variables, that can be located between array's variables, this makes the array "non-continuous" block of memory and makes navigation between cells more complicated, and thus, slower.

A temporary arrays is an object outside the PDV and can be organized as continuous block of memory that makes cell access truly direct and thus even faster. From the program code point of view only the `_TEMPORARY_` statement is something new, all other elements look like a regular SAS array.

code: temporary array

```
1 data _null_;
2   ARRAY tmp[1:5] $ 2 _TEMPORARY_ ("Pp" "Qq" "Rr" "Ss" "Tt");
3
4   key = 3;
5   data = tmp[key];
6   put data=;
7 run;
```

Program Data Vector

key	data	...	_N_	_ERROR_
3	Rr		1	0

tmp [char, 2]

[1]	[2]	[3]	[4]	[5]
Pp	Qq	Rr	Ss	Tt

the log

```
1 data=Rr;
```

WHAT'S THE POINT=?

You can easily observe that pointing to an array cell looks similar to use of the `SET` statement with the `POINT=` option. That's true, a data set's observations pointing can be used too (in fact, papers from the '80s are using this technique, see [**C. K. Ray, SUGI 1988**]). Disadvantage here is that they are disk stored, and we are also focusing on efficiency. Temporary arrays are memory resident, so all the I/O work is avoided, as long as the array's memory footprint fits in RAM.

You may think: "*Oh! Yes! The memory limitation...*" An array with one million of 128-bytes cells is less than 123MB... so please, stop... even with the default SAS' `MEMSIZE 2G` you have a plenty of room to work. And on modern computers, probably your web browser eats more RAM than that.

ASSOCIATIVE ARRAYS

As I mentioned in the introduction, the associative array was a synonym of hash table. I will use those names interchangeably but the further the text goes I will lean toward hash table.

An associative array is a dynamic data structure, that allows to access data (read/write) by providing key value. An associative array can grow or shrink dynamically during program execution. You can imagine or "intuitively" think about hash table as a bag for data that, without any particular ordering, stores your data and allow you access then through keys in very efficient, fast, way.

Since most programmers are familiar with "ordinary" arrays, I am going to use them for the introduction of hash tables. It is good to think about hash tables as a natural "expansion" of existing arrays' functionality. Having a "high-school teacher point of view" in mind, you could say: "*Temporary Array was my former student and had a very good performance in my classes and Hash Table, TA's younger sibling, just showed up first day on Campus.*"

As I mentioned earlier, temporary array is a separated (from the PDV) and reserved at the compilation phase (so by the definition, a fixed size) block of memory, divided into a fixed size cells of numeric or character type that can be accessed through an integer key (or index).

Similarly, a hash table, is an object separated from the PDF. In contrast to a temporary array, the hash table is initialized/instantiated in the execution phase so the piece of memory it gets can be dynamically re-sized while data are added or removed from the hash object.

As I wrote, the "basic" use case is a dictionary case, but hash table is much more than that. A single key can point not only to a single data cell, but to a whole "observation" of multiple cells (variables) of different types. Hash table allows to move even one step further, a single key can point to a group of "observations" (so not just a single row, but many!) Next extension, when compared with arrays, is that in case of hash tables the key does not have to be integer! The key can be any number, a character variable, or even be composed of several variables (key is sometimes called composed in such cases).

Yet another differentiation from array is the way how this (external to PDV) object interacts with Program Data Vector. Things like adding new elements to a hash table, verification if an element exist in it, declaration of the key or data portion, or replacing data assigned to a key. While working with hash tables to perform all those operations (and more) you are using so called methods and a dot notation. A method is an internal function associated with an object (here, a hash table) that execute a specific action on the object. The dot notation is a way of calling a method on a particular object. In case of an array you put square brackets to point a cell with the key, similarly the dot notation tells which method should be used on a given object. The general syntax is "*object dot method (arguments)*", for example a method allowing to verify if key of a given value exist in hash table `H` is the `.CHECK()` method, to see if 42 is a key you write: `rc = H.check(key:42)`. One more example comparing functions use and dot notation would be checking of array and hash table sizes. For an array `A` you would use the `DIM` function, e.g., `n = DIM(A);` and for a hash table the `.NUM_ITEMS` attribute would do the job, e.g. `n = H.ITEMS_NUM;`

Table 1 compares and contrasts properties of temporary arrays and hash tables.

SIDE-BY-SIDE COMPARISON WITH ARRAYS

The first contact with a hash table (associative array) will be a side-by-side comparison with a temporary table. Both arrays have integer key and character data portion of length 3 (bytes). Data values are sequential 3 letter sub-strings of the "ABCDEFGHIJKL" string. Both code snippets are built of three parts. The first initializes the data structure, the second populates it with data, the last looks up data for provided key values.

property	temporary array	hash table
key	integer	numeric, character or an arbitrary composition of those
sequential key values	yes	not required
ordering of key values	yes, forced at declaration	none, though ascending/descending ordering is possible to set with `ordered:` label
instantiation	compilation phase, `array` statement	execution phase, `declare hash` statement
key declaration	implicit	explicit, `.defineKey()` method
data portion declaration	implicit	explicit, `.defineData()` method
declaration end	none, `array` statement starts and ends the declaration	execution phase,`.defineDone()` method
data	single cell of a fixed type for one key value	one or more "observation" (build out of zero or more cells of different types) for one key value
size	fixed, set at the compilation phase	flexible, resizable at the execution phase
name	cannot be an existing PDF variable	cannot be an existing PDF variable
automatic data retain	yes	yes
key call	square brackets after the array name	through key's variables(s) in PDV
data read, add, or replace for a key	left- or right-side assignment operator (=)	with `.find()`, `.add()`, and `.replace()` methods
data loading from a SAS data set	explicit, with a loop	automatic at declaration or explicit with a loop
output to a SAS data set	explicit with a loop	automatic with `.output()` method or explicit with loop and hash iterator

Table 1. Properties of hash tables compared with temporary arrays.

code: array T

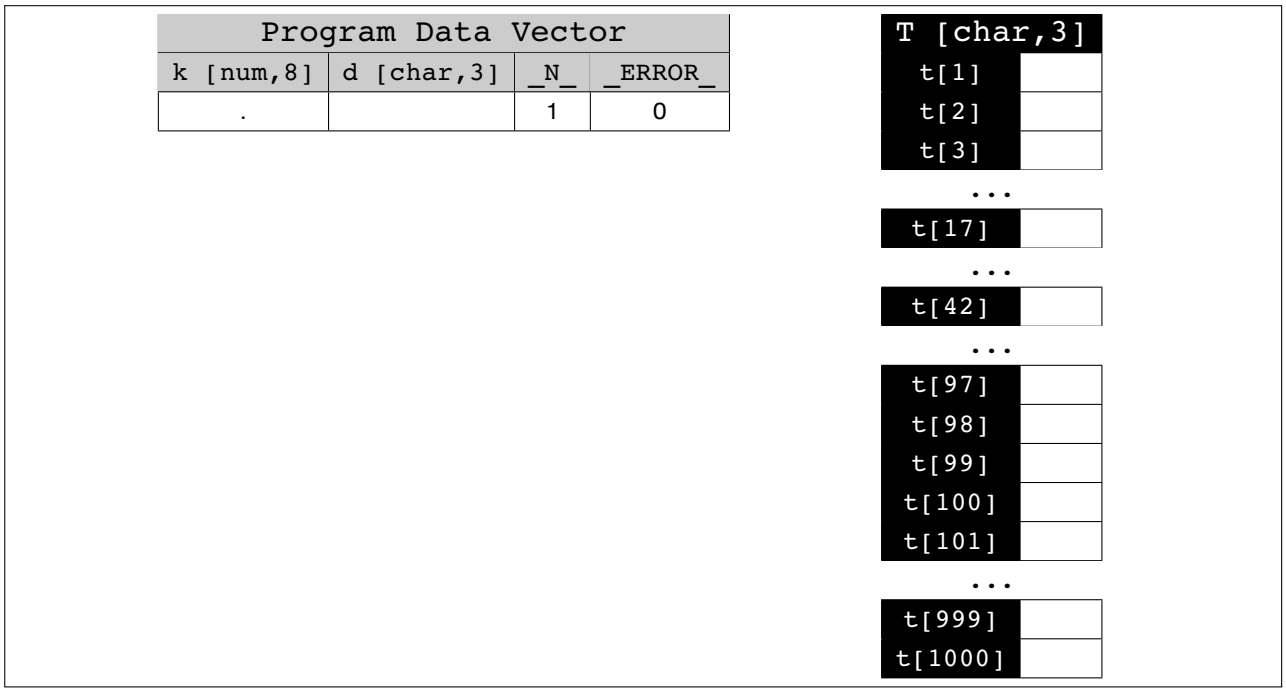
```
data _null_;
  length k 8 d $ 3;
  array T[1000] $ 3 _temporary_;
  do k = 1 to 100 by 2;
   d = substr("ABCDEFGHIJKL",
              mod(k,10)+1, 3);
   T[k] = d;
  end;
  do k = 1, 17, 42, 99;
    d = T[k];
    put k= d=;
  end;
run;
```

code: hash table H

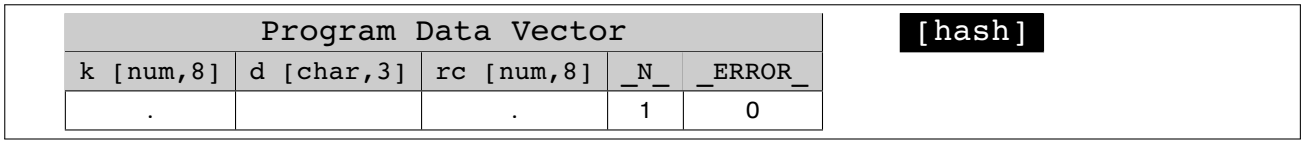
```
data _null_;
  length k 8 d $ 3;
  declare hash H(ordered:"A");
  H.defineKey("k");
  H.defineData("d");"
  H.defineDone();"
  do k = 1 to 100 by 2;
   d = substr("ABCDEFGHIJKL",
              mod(k,10)+1, 3);
    rc = H.add(key:k, data:d);
  end;
  do k = 1, 17, 42, 99;
    rc = H.find(key:k);
    if rc ^= 0 then d = " ";
    put k= d=;
  end;
run;
```

I will take you through this code in "chunks". In the first chunk you will see the creation of an object (lines 3 to 6), the second one (lines 8 to 12) is the data propagation, and the last (lines 14 to 18) is data search and extraction.

When the compilation phase of the left hand side code is done the memory looks like in the following figure. In the PDV you can see a numeric variable `k`, a character variable `d` of length 3, and technical variables `_N_` and `_ERROR_`. Also, all requested memory (1000 cells) for the array `T` is reserved.



When the compilation phase of the right hand side code is done the memory looks like in the following figure. In the PDV there are numeric variable `k`, character variable `d` of length 3, numeric variable `rc` (return code), and technical variables `_N_` and `_ERROR_`. For the hash table `H` only memory for a pointer localizing `H` is reserved.



When the execution phase starts statements instantiating the hash table `H` are run one after another.

The `DECLARE HASH` statement is a "two-fold" one. It worked in the compilation phase to declare a "special" variable of type *hash* named `H` and set the pointer to it. Now, because parenthesis were used, the `DECLARE HASH` statement also initializes the hash table `H` (the case "without" parenthesis is a subject of a separate, later, discussion). Additionally added (optional) *label* `ordered:"A"` forces the ascending order of the key values inside the hash table, this is not the default. By default, no ordering is kept inside the hash object, basically there is no need for that. Next, hash table identifier is set, also variables defining key part (`k`) and data part (`d`) are set. Those variables names are provided through `.DEFINEKEY()` and `.DEFINEDATA()` methods in form of a text expression (this time as a constant string expression). Those key and data variables are "connecting variables" associated with their counterparts in the PDV and are necessary for data transfer between hash object and the PDV. Since the initialization of hasht table variables names provided by `.DEFINEKEY()` and `.DEFINEDATA()` methods happens in the execution phase the PDV counterparts *have* to exist in the PDV beforehand. In case any of those variables does not exist in Program Data Vector an error is thrown in the execution of the DATA step. The error message is

```
ERROR: Undeclared key symbol for hash object at line X column Y.
ERROR: DATA STEP Component Object failure.  Aborted during the
EXECUTION phase.
```

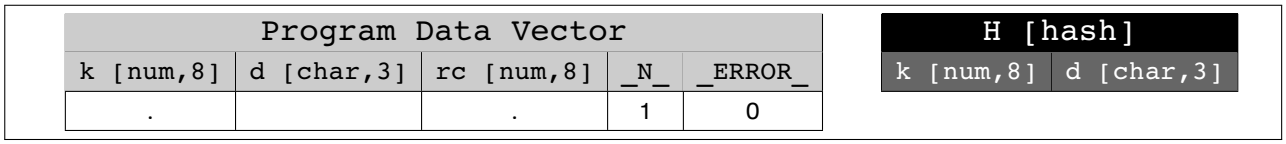
or

```
ERROR: Undeclared data symbol for hash object at line X column Y.
ERROR: DATA STEP Component Object failure.  Aborted during the
EXECUTION phase.
```

The simplest (not necessarily the most practical) way to avoid those errors is to use the `LENGTH` statement to define those required variables in the PDV.

The declaration concludes with the `.DEFINEDONE()` method. This one is kind like the semicolon in the `ARRAY` statement. If you try to use a hash table before the method is executed you will get `ERROR: Method defineDone must be called to complete initialization of hash object before line X column Y. ERROR: DATA STEP Component Object failure. Aborted during the EXECUTION phase.`

After the `.DEFINEDONE()` method runs the memory looks similar to the figure below.



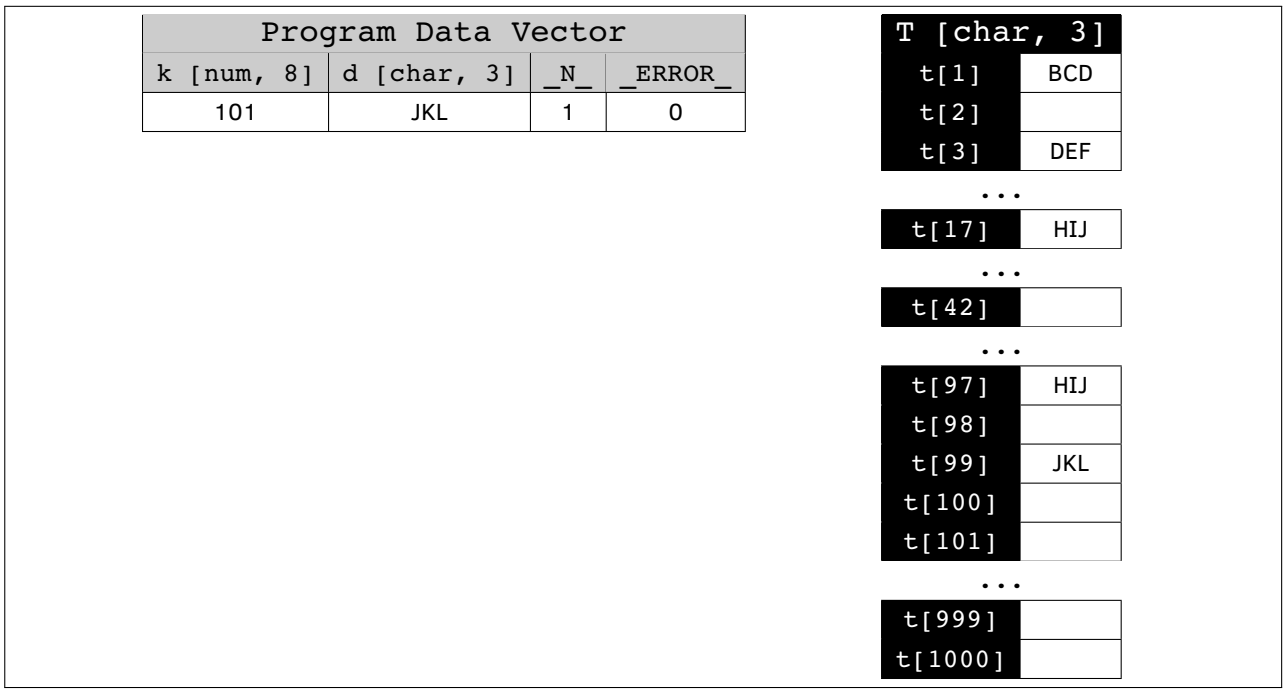
Program Data Vector

k [num,8]	d [char,3]	rc [num,8]	_N_	_ERROR_
.		.	1	0

H [hash]

k [num,8]	d [char,3]

When you go through lines 8 to 12, in both DATA steps, a loop over odd integers starting from 1 is executed. Every number gets a data portion that is a three bytes sub-string cut by the `SUBSTR` function, 1 gets "BCD", 3 gets "DEF", and so on. When the loop is done the state of memory for the left snippet looks like the figure below. It's worth noticing that not all cells reserved for the array were "used" (populated with data).



Program Data Vector

k [num, 8]	d [char, 3]	_N_	_ERROR_
101	JKL	1	0

T [char, 3]

t[1]	BCD
t[2]	
t[3]	DEF
...	
t[17]	HIJ
...	
t[42]	
...	
t[97]	HIJ
t[98]	
t[99]	JKL
t[100]	
t[101]	
...	
t[999]	
t[1000]	

The right hand side DATA step populates the hash table in the following way. In the first `DO-LOOP` iteration, for `k=1`, the `SUBSTR` function cuts the "BCD" and assigns the value to `d`. Next the `.ADD()` method is called to populate hash table with data from variables `k` and `d`. In the program the method is used in the form: `H.add(key:k, data:d)` that explicit indicates which PDV variables should be used. The `key:` *label* points the key variable (in general it can be an expression) and the `data:` *label* points the data variable. Values that are stored in `k` and `d` variables in the PDV, the moment of execution, are taken and added into the hash table. Usually (97% cases) you call the `.ADD()` method without explicit arguments pointing, in such

case SAS refers to the definition provide in the `.DEFINEKEY()` and `.DEFINEDATA()` methods and picks variables in PDV as pointed in those definition.

Successful execution of the `.ADD()` method results in adding new key and its data portion to the hash table and returns zero, Yes, zero! You can interpret this behavior the following way: the process went successfully thus there had to be 0 (zero) errors. The memory after the first iteration looks the following way:

Program Data Vector

k [num,8]	d [char,3]	rc [num,8]	_N_	_ERROR_
1	BCD	0	1	0

H [hash]

k [num,8]	d [char,3]
1	BCD

The second iteration updates `k` to 3, the `SUBSTR` function cuts the "DEF" and assigns the value to `d`, and the `.ADD()` adds the key and the data portion to `H` The memory looks the following way.

Program Data Vector

k [num,8]	d [char,3]	rc [num,8]	_N_	_ERROR_
3	DEF	0	1	0

H [hash]

k [num,8]	d [char,3]
1	BCD
3	DEF

Subsequent iterations expand the hash table `H` with new keys and data portions. In the example discussed all keys are unique (as in a dictionary) thus the `rc` variable is always equal to zero (the case with non-unique keys is discussed later). When the loop concludes the memory looks as follows

Program Data Vector

k [num,8]	d [char,3]	rc [num,8]	_N_	_ERROR_
101	JKL	0	1	0

H [hash]

k [num,8]	d [char,3]
1	BCD
3	DEF
5	FGH
...	
17	HIJ
...	
41	BCD
43	DEF
...	
95	FGH
97	HIJ
99	JKL

The last part of those two snippets (lines 14 to 18) presents data search and retrieval for a given list of keys. For temporary array the values of variable `k` are used in square brackets to point the location in the array `T`, and the assignment (=) statement puts the result in `d`. The LOG clearly shows it

the log - array

```
1  ...
2  14      do k = 1, 17, 42, 99;
3  15        d = T[k];
4  16
5  17        put k= d=;
6  18      end;
7  19    run;
8
9  k=1 d=BCD
10 k=17 d=HIJ
11 k=42 d=
12 k=99 d=JKL
13 NOTE: DATA statement used (Total process time):
```

```
      real time              0.00 seconds
      cpu time               0.00 seconds
```

The printout for the hash table case, as expected, is identical, but the work executed "under the hood" is a bit different than the array case.

the log - hash table

```
...
14      do k = 1, 17, 42, 99;
15        rc = H.find(key:k);
16        if rc ^= 0 then d = " ";
17        put k= d=;
18      end;
19    run;

k=1 d=BCD
k=17 d=HIJ
k=42 d=
k=99 d=JKL
NOTE: DATA statement used (Total process time):
      real time              0.00 seconds
      cpu time               0.00 seconds
```

To retrieve data for a given key value, the `.FIND()` method is used. In the example an explicit information from where the value of key should be taken is provided, i.e. `rc = H.find(key:k);`, similar to the `.ADD()` method used earlier. A successful key search of the `.FIND()` method results in two things, first, it returns zero (the "zero errors" code of success), second and the foremost, the method overrides PDV variable `d` with data value located in its hash table counterpart `d` associated with the key. If the value of the key is not found the value of data variable `d` in the PDV is not replaced/changed/modified, it stays as it was before the method execution, thus the `IF-THEN` statement is needed. If the `rc ^= 0` condition is satisfied, what means "there was a problem" (the key was not found), then `d` in PDV is set to missing (you do not want to carry wrong valuer from the loop's previous iteration forward). Currently the code has two instructions: search and "conditional" missing, but it can be reduced to a single instruction.

code

```
...
  do k = 1, 17, 42, 99;
    if h.find(key:k) then call missing(d);

    put k= d=;
  end;
run;
```

The code can be written this way because the `.FIND()` method returns value and (in case of success) overrides `d` in the PDV at the same time, and it only returns non-zero value and does not touch `d` in case of failure. The `CALL MISSING` subroutine is used to make the code variable type independent.

As you have seen in the summary table above, the hash table allows you to use not only integers as keys but also other types of data, for example the following code snippet creates a hash table that uses character variable for key and numeric one for data.

code: character key

```
data _null_;
  declare hash gadabouts();
  gadabouts.defineKey("k");
  gadabouts.defineData("d");
  gadabouts.defineDone();
```

```
6
7    do k = "Alf", "BoB", "Cid", "Dex", "Edd"; /* key  */
8      d+100;                                  /* data */
9      rc = gadabouts.add();
10   end;
11
12   do k = "BoB", "Zed", "Edd";
13     if gadabouts.find(key:k) then call missing(d);
14     put k= d=;
15   end;
16 run;
```

the log - character key

```
1 k=BoB d=200
2 k=Zed d=.
3 k=Edd d=500
```

This is more or less all you need to know to satisfy the 80/20 rule and use the hash object just as a dictionary. The article could end here, but no... I am sure you want to learn more!

THE DEVIL'S IN THE DETAILS

In Polish language we say "*im dalej w las, tym więcej drzew*" what literally translates to "the deeper into the woods, the more trees you see", and the meaning (as you can guess) is the more you get into something, the more complicated it becomes. If you want to explore that remaining 20% of hash tables use cases, expanding above the "just a dictionary" part, this and further sections are for you.

DECLARATION

Let's look one more time at the hash table declaration and initialization process and its components, i.e. the `DECLARE HASH` statement and `.DEFINEKEY()`, `.DEFINEDATA()`, and `.DEFINEDONE()` methods.

The `DECLARE HASH` statement can be used in two possible ways. The first, where you *do not* add parenthesis after the hash table name

```
declare hash H;
```

instructs SAS to *only* create a system variable of type *hash*, that in the future will store a pointer to an instance of object `H`. When you decide this path you would have to instantiate `H` in the next step with use of the `_NEW_` statement

```
H = _NEW_ hash(<label1:value1, ..., labelN:valueN>);
```

The second way, where you *do* add parenthesis after the hash table name

```
declare hash H(<label1:value1, ..., labelN:valueN>);
```

instructs SAS to both declare and initiate hash object `H`. In other words those two declarations in programs below are "practically" equivalent

code: practical equivalence

```
1 data _null_;
2   declare hash H;
3   H = _new_ hash(ordered:"A");
4
5   declare hash G(ordered:"A");
6 run;
```

At the first sight it may seem redundant to have two tools for the same job (especially the "longer" one), but in upcoming sections you will learn why there is a point in keeping both.

LABELS

Regardless of which of those two approaches you decide to use it is a good ideal to learn a bit about those *labels* mentioned in the definition, or at least the most popular ones. *An important and general remark* about labels (in all methods, not only in declaration) is that the value of the label (after the colon) can be a string constant (like in the example above), a character variables containing the value, or an expression that evaluated resolves to the label value.

Ordered - This label orders keys. By default keys in a hash table are not ordered in any particular way. If you ask SAS to list them one after another from the first to the last you would see they are kind of random. This is caused by the fact that keys values are irrelevant to internal ordering of the hash object. Obviously the situation where ordering does matter for a programmer can happen, in such case the `ordered:` label can force ordering. With any of the: "A", "Ascending", "Y", and "Yes" the order of keys will be ascending, with "D" or "Descending" the order of keys will be set to descending, and case with no label or "N" or "No" sets the default behavior of no ordering at all. The example below shows all three cases:

code: ordering

```
data _null_;
  length k 8 d $ 1;
  declare hash hA(ordered:"A");
  hA.defineKey("k");
  hA.defineData("d");
  hA.defineDone();

  declare hash hD(ordered:"D");
  hD.defineKey("k");
  hD.defineData("d");
  hD.defineDone();

  declare hash hN(ordered:"N");
  hN.defineKey("k");
  hN.defineData("d");
  hN.defineDone();

  do k = 1,2,3;
   d = substr("ABC", k, 1);
   rc = hA.add(key:k, data:d);
   rc = hD.add(key:k, data:d);
   rc = hN.add(key:k, data:d);
  end;
run;
```

Hash tables `HA`, `HD`, and `HN` are presented below (if you wonder how I checked that continue reading to the next sections).

HA [hash]	
k [num,8]	d [char,1]
1	A
2	B
3	C

HD [hash]	
k [num,8]	d [char,1]
3	C
2	B
1	A

HN [hash]	
k [num,8]	d [char,1]
2	B
3	C
1	A

Dataset - This label points data source to be loaded into the hash table. The `dataset:` allows you to point a data set from which you can automatically load data into a hash table. Data set options are honored! To

use the label you write code like the following:

```
declare hash HashTableName(dataset:  expression);
```

The `expression`, as I mentioned earlier is a character constant, variable or character expression that resolves to the SAS data set you want to use. Lets say we have data set `work.X`

code: data set work.X

```
data work.X;
  input a b x;
cards;
1 2 1
3 4 1
5 6 0
7 8 0
;
run;
```

Use of `work.X` to populate hash table can be like in the following code snippet, mind use of the `WHERE=` data set option!

code: dataset

```
data _null_;
  length a b 8;
  declare hash H(dataset:"work.X(where=(x=1))");
  h.defineKey("a");
  h.defineData("b");
  h.defineDone();
run;
```

In the compilation phase lengths of variables `a` and `b` are set and then during execution phase the initialized hash table `H` is populated with data from the data set `work.X` (filtered by the `x=1` condition). With respect to the ordering the memory state looks like below.

Program Data Vector			
a [num,8]	b [num,8]	_N_	_ERROR_
.	.	1	0

H [hash]	
a [num,8]	b [num,8]
1	2
3	4

From practical point of view, in terms of data propagation into hash table, code above is equivalent with the snippet below where first, the hash table is declared and then, in an explicit DOW-loop data are read form the data set and added with the `.ADD()` method.

code: direct data propagation

```
data _null_;
  length a b 8;
  declare hash H();
  h.defineKey("a");
  h.defineData("b");
  h.defineDone();
  do until(end);
    set work.X end=end;
    where x = 1;
    h.add(key:a, data:b);
  end;
stop;
run;
```

Though they are similar in behavior, those two programs have some subtle differences worth discussing. The first difference is: after execution, the first program produces two ugly looking notes

the log - ugly notes

```
NOTE: Variable a is uninitialized.
NOTE: Variable b is uninitialized.
```

while the second does not. The reason for that is the fact that during "automatic" data loading into hash table no data goes through variables `a` and `b` in the PDV, thus `a`'s and `b`'s values are never really populated (the `LENGTH` statement is active only in compilation phase). In the second DATA step the `SET` statement reads data into PDV before they are added into the hash table.

The second difference is a direct consequence of the `SET` statement use. The `SET` statement makes variables `a` and `b` implicitly retained, whence in the first DATA step at the top of iteration of the DATA step loop `a` and `b` are set to missing and in the second DATA step they values stay unchanged.

Read further to find out how to resolve those "inconveniences", but for now...

The third difference. In case when the `dataset:` label is used, the list of variables that can be used in the key part or in the data part of the hash table definition is limited *only* to variables from the selected data set. This means that if you modify the code to add a new variable `c` (external to the data set) to hash table data portion, like in the code below you will end up with error.

code: new variable

```
data _null_;
  length a b c 8;
  c = 42;
  declare hash h(dataset:"work.X");
  h.defineKey("a");
  h.defineData("b", "c");
  h.defineDone();
run;
```

The LOG looks like the following.

the log - failed result

```
...
7       h.defineDone();
8     run;

NOTE: Variable a is uninitialized.
NOTE: Variable b is uninitialized.
ERROR: Variable c is not on file WORK.X.
ERROR: Hash data set load failed at line 7 column 3.
ERROR: DATA STEP Component Object failure. Aborted during the EXECUTION phase.
NOTE: The SAS System stopped processing this step because of errors.
NOTE: DATA statement used (Total process time):
      real time           0.00 seconds
      cpu time            0.00 seconds
```

While in the second case everything works just fine.

code: new variable

```
data _null_;
  length a b c 8;
  c = 42;
  declare hash h();
  h.defineKey("a");
  h.defineData("b", "c");
```

```
  h.defineDone();

  do until(end);
    set  work.X end=end;
    h.add(key:a, data:b, data:c);
  end;
run;
```

On one hand, the behavior is a bit limiting-*ish*, on the other hand keeps the hash table tidy from unwanted content.

The ugly "uninitialized" note can be easily removed in a few simple ways. The first one combines the `LENGTH` statement and the `CALL MISSING` subroutine like in the following example

code: solving uninitialized variables issue

```
data _null_;
  length a b 8;
  call missing(a, b);
  declare hash h(dataset:"work.X");
  h.defineKey("a");
  h.defineData("b");
  h.defineDone();
run;
```

One flaw of this approach is hard-coding, because you provide metadata typed by hand. The second flaw is the impracticality because if metadata change you will have to manually fix the code. That is why an alternative approach that uses conditional `SET` statement seems to be more suitable. The solution is based on combination of *always* false `IF-THEN` and `SET` statements. The `if 0 then ...` part excludes the `SET` statement from the execution phase (the false condition never branches condition to reading data) at the same time, during compilation phase, `SET` statement provides all required metadata. Just like in the following program

code: conditional SET statement

```
data _null_;
  if 0 then SET work.X;
  declare hash H(dataset:"work.X");
  h.defineKey("a");
  h.defineData("b");
  h.defineDone();
stop;
run;
```

The only thing you have to remember is to add the `STOP` statement just to avoid another ugly note that `NOTE: DATA STEP stopped due to looping`. Of course, if you have in your code some other `SET` statement that guarantees a natural stop condition, then you do not need to call an explicit stop.

Multidata - This label allows keys to have multiple "observations" assigned to them. If you set `multidata:"Y"` (or "YES") a single key value in the hash table can have multiple data portions attached to it (this clearly moves your hash tables use cases beyond a dictionary). Default value ("N"/"NO") keeps standard, one key - one data, dictionary style. In the following code every key value is associated with two data portions

code: multidata

```
data _null_;
  declare hash h(multidata:"Y");
  h.defineKey("k");
```

```
  h.defineData("d");
  h.defineDone();

  k = 1; d = "ABC";
  rc=h.add();
  put _all_;

  k = 2; d = "DEF";
  rc=h.add();
  put _all_;

  k = 1; d = "GHI";
  rc=h.add();
  put _all_;

  k = 2; d = "JKL";
  rc=h.add();
  put _all_;
run;
```

The logical memory representation looks like the following figure.

Program Data Vector

k [num,8]	d [char,3]	rc [num,8]	_N_	_ERROR_
2	JKL	0	1	0

H [hash]

k [num,8]	d [char,3]
1	ABC
	GHI
2	DEF
	JKL

The printout in the LOG with `rc=0` after every `.ADD()` method execution confirms that all data are successfully added

the log - multidata

```
...
21     put _all_;
22   run;

k=1 d=ABC rc=0 _ERROR_=0 _N_=1
k=2 d=DEF rc=0 _ERROR_=0 _N_=1
k=1 d=GHI rc=0 _ERROR_=0 _N_=1
k=2 d=JKL rc=0 _ERROR_=0 _N_=1
NOTE: DATA statement used (Total process time):
      real time           0.00 seconds
      cpu time            0.00 seconds
```

In case when the `multidata:` label had not been used the memory and LOG would have looked as follows.

Program Data Vector

k [num,8]	d [char,3]	rc [num,8]	_N_	_ERROR_
2	JKL	160036	1	0

H [hash]

k [num,8]	d [char,3]
1	ABC
2	DEF

the log - no multidata

```
...
21     put _all_;
22   run;

k=1 d=ABC rc=0 _ERROR_=0 _N_=1
```

```
k=2 d=DEF rc=0 _ERROR_=0 _N_=1
k=1 d=GHI rc=160036 _ERROR_=0 _N_=1
k=2 d=JKL rc=160036 _ERROR_=0 _N_=1
NOTE: DATA statement used (Total process time):
      real time           0.00 seconds
      cpu time            0.00 seconds
```

A fact worth noting is the following one: though keys are not ordered in any particular way, data portion, in case of multiple data elements, are always ordered in the sequence they were added to the hash table! You can see this behavior in the next code snippet. Just as curiosity, `multidata:` label works also in combination with the `dataset:` label.

code: multidata & dataset

```
data dupKeys;
  k = 1; d = "ABC"; output;
  k = 2; d = "DEF"; output;
  k = 1; d = "GHI"; output;
  k = 2; d = "JKL"; output;
run;

data _null_;
  if 0 then SET dupKeys;
  declare hash H(dataset:"dupKeys", multidata:"Y");
  h.defineKey("k");
  h.defineData("k", "d");
  h.defineDone();

  h.output(dataset:"work.MultiData");
  stop;
run;
```

After execution the memory and LOG looks as follows.

Program Data Vector

k [num,8]	d [char,3]	_N_	_ERROR_
.	.	1	0

H [hash]

k [num,8]	k [num,8]	d [char,3]
1	1	ABC
	1	GHI
2	2	DEF
	2	JKL

the log

```
...
15          h.output(dataset:"work.MultiData");
16          stop;
17        run;

NOTE: There were 4 observations read from the data set WORK.DUPKEYS.
NOTE: The data set WORK.MULTIDATA has 4 observations and 2 variables.
NOTE: DATA statement used (Total process time):
      real time           0.00 seconds
      cpu time            0.01 seconds
```

As you observed there was one more `dataset:` label highlighted in the snippet. The second `dataset:` label is related to the `.OUTPUT()` method (mentioned already in the Table 1). "By the book" approach would

be to just mention the method here and say it will be discussed in more details in the section dedicated to methods. But in my opinion, it is too fundamental one. So, please forgive me, but I'll break the rhythm...

The `.OUTPUT()` method allows (as the name suggests) to output content of a hash table into a SAS data set. The method allows to create new SAS data sets during execution phase *without*(!) necessity of listing them in the `DATA` statement! This gives a plenty of new programming opportunities! For example, data sets can be created conditionally. Or for example, since the label's value can be a character expression, the data set name can be created dynamically, e.g., with change of the output library. The following program exemplifies the discussion.

code: output method

```
data _null_;
  declare hash H(multidata:"Y");
  h.defineKey("k");
  h.defineData("d");
  h.defineDone();

  do i=1 by 1 until(end);
    set dupKeys end=end;
    rc = h.add();
  end;
  put i=;

  /* "conditional" .output() */
  if i < 42 then h.output(dataset:"work.conditional");

  /* "dynamic" .output() */
  h.output(dataset:"work.dynamic" || put(today(), yymmddn8.) );
  stop;
run;
```

In the LOG it is clearly visible that both conditional execution and dynamic name creation work just fine.

the log

```
...
13          /* "conditional" .output() */
14          if i < 42 then h.output(dataset:"work.conditional");
15
16          /* "dunamic" .output() */
17          h.output(dataset:"work.dunamic" || put(today(), yymmddn8.) );
18          stop;
19        run;

i=4
NOTE: The data set WORK.CONDITIONAL has 4 observations and 1 variables.
NOTE: The data set WORK.DYNAMIC20261022 has 4 observations and 1 variables.
NOTE: There were 4 observations read from the data set WORK.DUPKEYS.
NOTE: DATA statement used (Total process time):
      real time           0.00 seconds
      cpu time            0.00 seconds
```

Analysis of LOG notes reveals one more very important fact about hash tables behavior. Though we used `k` and `d` in the hash table declaration, after execution of the `.OUTPUT()` method both data sets have only one variable! It is not a coincidence or an accident, but this will be discussed during detailed description of the `.DEFINEKEY()` and `.DEFINEDATA()` methods.

Allow me to combine two final remarks about the `.OUTPUT()` method in one paragraph. The first is: the `.OUTPUT()` method allows multiple `dataset:` labels, so multiple data sets can be created with single call. The second is: `dataset:` labels allow for data set option also in the `.OUTPUT()` method. The snippet below combines both features.

code: output method, cont.

```
data _null_;
  declare hash H();
  h.defineKey("k");
  h.defineDone();

  do k=1 to 9;
    rc = h.add();
  end;

  h.output(dataset:"work.even(where=(mod(k,2)=0))"
          ,dataset:"work.odd(where=(mod(k,2)=1))");
run;
```

the log

```
...
10     h.output(dataset:"work.even(where=(mod(k,2)=0))"
11             ,dataset:"work.odd(where=(mod(k,2)=1))");
12   run;

NOTE: The data set WORK.EVEN has 4 observations and 1 variables.
NOTE: The data set WORK.ODD has 5 observations and 1 variables.
NOTE: DATA statement used (Total process time):
      real time           0.06 seconds
      cpu time            0.01 seconds
```

Ok, let's go back to the main subject, i.e., labels continued.

Duplicate - Handles policy for duplicated data portions for a given key value. The default behavior of a hash table is to ignore duplicated data portion for a key value during data loading. The `duplicate:` label is designed to verify if duplicated data portions exist and how to handle them. When the `duplicate:` label is used without the `dataset:` label a WARNING: Hash Object DATASET option should be used when specifying the DUPLICATE option is produced in the LOG. The `duplicate:` label allows two policies for duplicated data handling, the first, with "R"/"Replace" value, forces existing data portion to be replaced with a new one, the second, with "E"/"Error" value, breaks the loading process with a spectacular error in the LOG. In the next code snippet, you can see those two behaviors.

code: duplicate

```
data _null_;
  if 0 then SET dupKeys;
  declare hash h(dataset:"dupKeys", duplicate:"R");
  h.defineKey("k");
  h.defineData("k", "d");
  h.defineDone();

  h.output(dataset:"work.R");
  stop;
run;

data _null_;
  if 0 then SET dupKeys;
  declare hash h(dataset:"dupKeys", duplicate:"E");
```

```
  h.defineKey("k");
  h.defineData("k", "d");
  h.defineDone();

  h.output(dataset:"work.E");
  stop;
run;
```

The LOG looks like the following.

the log - duplicate

```
...
8             h.output(dataset:"work.R");
9             stop;
10          run;

NOTE: There were 4 observations read from the data set WORK.DUPKEYS.
NOTE: The data set WORK.R has 2 observations and 2 variables.
NOTE: DATA statement used (Total process time):
      real time           0.00 seconds
      cpu time            0.01 seconds

...
19            h.output(dataset:"work.E");
20            stop;
21          run;

ERROR: Duplicate key found when loading data set dupKeys at line 17 column 3.
NOTE: There were 3 observations read from the data set WORK.DUPKEYS.
ERROR: Hash data set load failed at line 17 column 3.
ERROR: DATA STEP Component Object failure.  Aborted during the EXECUTION phase.
NOTE: The SAS System stopped processing this step because of errors.
NOTE: DATA statement used (Total process time):
      real time           0.00 seconds
      cpu time            0.00 seconds

```

OK. The labels list goes on but now stop. The other labels will be presented in upcoming sections.

KEYS AD DATA

When a hash table is declared most often (but not always) two methods, the `.DEFINEKEY()` and the `.DEFINEDATA()`, are used. It is worth knowing they also exhibit some specific behaviors.

The first one is: the `.DEFINEKEY()` method is obligatory! If you try to create hash table without any key, you will get an error.

code: no key

```
data _null_;
  length k 8 d $ 3;
  declare hash h();
  h.defineData("k", "d");
  h.defineDone();
run;
```

The LOG confirms the behavior.

the log

```
...
5       h.defineDone();
6     run;

NOTE: Variable k is uninitialized.
NOTE: Variable d is uninitialized.
ERROR: Uninitialized keys for hash object at line 5 column 3.
ERROR: DATA STEP Component Object failure. Aborted during the EXECUTION phase.
NOTE: The SAS System stopped processing this step because of errors.
NOTE: DATA statement used (Total process time):
      real time           0.00 seconds
      cpu time            0.00 seconds
```

The second: the `.DEFINEDATA()` *is not* mandatory, but... if you decide *not* to use it *all* key variables are automatically(!) assigned to the data portion. The code like one below, though only the `.DEFINEKEY()` method is used, "inflates" memory use.

code: no defineData

```
data _null_;
  length k 8 d $ 3;
  declare hash h();
  h.defineKey("k", "d");
  h.defineDone();

  k = 1; d = "ABC";
  rc = h.add();
  k = 2; d = "DEF";
  rc = h.add();

  h.output(dataset:"work.H");
run;
```

The LOG and memory looks like the following.

the log

```
...
12            h.output(dataset:"work.H");
13          run;

NOTE: The data set WORK.H has 2 observations and 2 variables.
NOTE: DATA statement used (Total process time):
      real time           0.00 seconds
      cpu time            0.01 seconds
```

The memory just before `.OUTPUT()` method execution.

Program Data Vector

k [num,8]	d [char,3]	_N_	_ERROR_
2	DEF	1	0

H [hash]

k [num,8]	d [char,3]	k [num,8]	d [char,3]
1	ABC	1	ABC
2	JKL	2	JKL

It is especially important to remember about this behavior when the key is heavy, i.e., is composed of many variables, or have wide variables. And the following simple example shows that in shortage of memory cases an "artificial" data portion can save you some space.

code

```
options fullstimer;
data _null_;
  length k1 k2 k3 k4 k5 k6 8;
  declare hash h();
  h.defineKey("k1", "k2", "k3", "k4", "k5", "k6"); /* composed key */
  h.defineDone();

  do _N_ = 1 to 1e6;
    k1 = _N_; k2 = _N_; k3 = _N_; k4 = _N_; k5 = _N_; k6 = _N_;
    rc = h.add();
  end;
  h.output(dataset:"work.H1");
run;

data _null_;
  length k1 k2 k3 k4 k5 d 8;
  declare hash h();
  h.defineKey("k1", "k2", "k3", "k4", "k5", "k6");
  h.defineData("d");
  d = .; /* "fake" data part */
  h.defineDone();

  do _N_ = 1 to 1e6;
    k1 = _N_; k2 = _N_; k3 = _N_; k4 = _N_; k5 = _N_; k6 = _N_;
    rc = h.add();
  end;
  h.output(dataset:"work.H2");
run;
```

In the LOG you can see that the "updated" memory usage went down to about 70% of the starting value

the log

```
 1          options fullstimer;
...
 13           h.output(dataset:"work.H1");
 14         run;

 NOTE: The data set WORK.H1 has 1000000 observations and 6 variables.
 NOTE: DATA statement used (Total process time):
       real time           0.61 seconds
       user cpu time       0.52 seconds
       memory              115283.87k
       OS Memory           155048.00k
...
 29           h.output(dataset:"work.H2");
 30         run;

 NOTE: The data set WORK.H2 has 1000000 observations and 1 variables.
 NOTE: DATA statement used (Total process time):
       real time           0.61 seconds
       user cpu time       0.55 seconds
       memory              75920.53k
       OS Memory           110248.00k
```

It is important to remember that the .OUTPUT() method produces data set work.H1 with 6 variables, but the work.H2 data set will have only one. A property that is yet another "specificity" is: when the .DEFINEDATA() is used, only the variables listed in the method are outputted to a SAS data set by the .OUTPUT() method. In other words, key variables not listed also as data portion variables are ignored. Methods retrieving data from a hash table, like for example the .FIND() method, are affected too, they simply do not update key variables counterparts in the PDV. This is an example reflecting the behavior.

code

```
data _null_;
  length k 8 d $ 3;
  declare hash h();
  h.defineKey("k");
  h.defineData("d");
  h.defineDone();

  k = 1; d = "ABC";
  rc = h.add();
  k = 2; d = "DEF";
  rc = h.add();

  put "1) " _all_;
  call missing(of _all_);
  put "2) " _all_;
  rc = h.find(key:1); /* a constant passed as the key value */
  put "3) " _all_;    /* but k in the PDV stays missing     */
run;
```

"Clean up" of the PDV is done with the CALL MISSING subroutine, but then the .FIND() method (with key value provided as a constant) updates only data portion variables in the PDV.

the log

```
...
16     rc = h.find(key:1); /* a constant passed as the key value */
17     put "3) " _all_;    /* but k in the PDV stays missing     */
18   run;

1) k=2 d=DEF rc=0 _ERROR_=0 _N_=1
2) k=. d=    rc=. _ERROR_=0 _N_=1
3) k=. d=ABC rc=0 _ERROR_=0 _N_=1
NOTE: DATA statement used (Total process time):
      real time           0.00 seconds
      cpu time            0.01 seconds
```

Thus, if you need key variables also as "data" you have to list those variables in the .DEFINEDATA().

But... (of course there is a but!) if you decide that data portion is not declared and you still want to use explicit key: label in the .ADD() method, you also have to use the data: label In other words, in the example below, the first data step fails and the other works perfectly fine.

code

```
data _null_;
  retain k 0;
  declare hash h();
  h.defineKey("k");
  h.defineDone();

  x = 17;
  rc = h.add(key:x);
run;
```

```
data _null_;
  retain k 0;
  declare hash h();
  h.defineKey("k");
  h.defineDone();

  x = 42;
  rc = h.add(key:x, data:x);
run;
```

the log

```
...
8       rc = h.add(key:x);
9     run;

ERROR: Incorrect number of data entries specified at line 8 column 8.
ERROR: DATA STEP Component Object failure. Aborted during the EXECUTION phase.
NOTE: The SAS System stopped processing this step because of errors.
NOTE: DATA statement used (Total process time):
      real time           0.00 seconds
      cpu time            0.01 seconds
...
18      rc = h.add(key:x, data:x);
19    run;

NOTE: DATA statement used (Total process time):
      real time           0.00 seconds
      cpu time            0.01 seconds
```

Let me point out a little crazy side note. Consequence of the above is that a peculiarity as the one in code below can show up in your head.

code: I beg you, don't...

```
data _null_;
  retain k 0;
  declare hash h();
  h.defineKey("k");
  h.defineDone();

  x = 42;
  rc = h.add(key:x, data:17);

  rc = h.output(dataset:"have"); /* ?? */

  rc = h.find(key:42);           /* ?? */
  put rc = ;
run;
```

Since data portion is not used the key is automatically expanded to it; but since the `.ADD()` method allows to distinguish what key and data values will be pushed into hash object; in the result you can have a "paradox code" like the one above, that seems to produce a hash table that "*have only one(!) key but with two(?) different values...*" Of course, with the explanation from previous paragraphs, you know everything is fine. Code like the one above would be a great test question in the `sasensei` quiz game, unfortunately `sasensei` is not active anymore, so the only other occasion to use it would be a nerdy SAS conference cocktail party... But I am begging you do not write code like that in production... The fact you can does not mean you should...

OK. Now let's get back to some useful things.

The next property touches both the `.DEFINEKEY()` and the `.DEFINEDATA()` methods. In previous examples, whenever there were multiple data or composite keys the list of variables was provided as a comma-separated list of expressions (constants in most examples). It is not the only way to create such lists. The `.DEFINEKEY()` and the `.DEFINEDATA()` method were designed so that until the `.DEFINEDONE()` method is executed, every call to `.DEFINEKEY()` and `.DEFINEDATA()` adds new values to the data or keys list and expands it. In the code snippet both methods are executed multiple times to add values to corresponding lists. The `.DEFINEKEY()` executes three times and the `.DEFINEDATA()` runs in a loop with an expression using the `VNAME` function to retrieve variables names.

code: multiple calls

```
data _null_;
  length k1 k2 k3 8 d1 - d10 $ 4;
  array dd d1 - d10 (10 * "abcd");

  declare hash H();

  h.defineKey("k1");
  h.defineKey("k2");
  h.defineKey("k3");

  h.defineData("k1");
  h.defineData("k2");
  h.defineData("k3");
  do over dd;
    h.defineData(vname(dd));
  end;
  h.defineDone();

  k1 = 1; k2 = 2; k3 = 3;

  rc = h.add();

  rc = h.output(dataset:"thirteen");
run;
```

The LOG confirms successful execution.

the log

```
...
23     rc = h.output(dataset:"thirteen");
24   run;

NOTE: The data set WORK.THIRTEEN has 1 observations and 13 variables.
NOTE: DATA statement used (Total process time):
      real time           0.11 seconds
      cpu time            0.04 seconds
```

Another practical functionality is a "wholesale" variables selection. When you are using the `dataset:` label to get data from a particular data set, both methods give you opportunity to ask for all variables at one time. The `all:"yes"` label does the job. You call it the following way:

`.defineKey(all:"yes");` or `.defineData(all:"yes");`

This way you can skip labor-some process of listing variables with one command.

I HAVE HASH TABLE, WHAT CAN I DO WITH IT?

Now, when you know how to declare and instantiate a hash table, let's focus on other methods that can be used in a "real world" programming. In this section you will learn (or refresh your knowledge) about how to add, modify, remove, or retrieve data from a hash table. You have already read about the `.ADD()` and the `.FIND()` methods. Both methods, when used with empty parenthesis, will communicate with PDV through "as defined" keys and data variables, i.e., if you executed `.defineKey("k");` and `.defineData("d");`, then `.FIND()` will search for key value in the `k` variable and will put data (if key is found) into the `d` variable. If you decide to use `key:` and `data:` labels, in such case you must remember that *all* keys and data portion variables have to be listed and in *exactly the same order* as in the declaration. Both methods return value 0 (zero) when executed successfully and something non-zero when they fail, that is why it is a good practice to assign them to a return code variable (e.g., `rc`).

In particular, the `.ADD()` method returns non-zero value when you try to add already existing key to a hash table. Like in the example below.

code: rc

```
data _null_;
  length k 8;
  declare hash H();
  h.defineKey("k");
  h.defineDone();

  k = 1;
  h.add();
  k = 1;
  h.add();

  h.output(dataset:"h1");
run;

data _null_;
  length k 8;
  declare hash H();
  h.defineKey("k");
  h.defineDone();

  k = 1;
  rc = h.add();
  k = 1;
  rc = h.add();

  h.output(dataset:"h2");
run;
```

The LOG tells you everything.

the log

```
...
12     h.output(dataset:"h1");
13   run;

ERROR: Duplicate key.
NOTE: The data set WORK.H1 has 1 observations and 1 variables.
NOTE: The SAS System stopped processing this step because of errors.
NOTE: DATA statement used (Total process time):
      real time           0.07 seconds
      cpu time            0.03 seconds

```

```
...
26      h.output(dataset:"h2");
27    run;

NOTE: The data set WORK.H2 has 1 observations and 1 variables.
NOTE: DATA statement used (Total process time):
      real time           0.06 seconds
      cpu time            0.01 seconds
```

Of course, when the `multidata:` label (that overrides the default "uniqueness of the key" condition) is used no error, even without the `rc` variable, will be shown.

The `.FIND()` method also returns zero when it is successfully executed and non-zero otherwise. When key is found the `.FIND()` method automatically picks up data portion associated to the key and puts it into appropriate PDV variable.

In some cases, you just need to verify if a key exists in the hash object and the additional data "transport" between hash and PDF seems to be a waste of resources. In this situation instead of finding you can just check. The simplest way to do this, i.e., verify key's existence only, is the `.CHECK()` method. Similarly, as two previous methods, zero (meaning "*I have zero problems with finding the key you have asked about*") is returned when the key exists in the hash table. The example below summarizes it all.

code: check

```
data _null_;
  length k d 8;
  declare hash H();
  h.defineKey("k");
  h.defineData("d");
  h.defineDone();

  do i = 1 to 9 by 2
    rc = h.add(key:i, data:(10*i));
  end;
  call missing(i,k,d,rc);

  do k = 1,2,3,4,5,6,7,8,9;
    rc = h.check();
    put k= d= rc=;
  end;
run;
```

log przedstawia się następująco

the log

```
...
13      do k = 1,2,3,4,5,6,7,8,9;
14        rc = h.check();
15        put k= d= rc=;
16      end;
17    run;

k=1 d=. rc=0
k=2 d=. rc=160038
k=3 d=. rc=0
k=4 d=. rc=160038
k=5 d=. rc=0
k=6 d=. rc=160038
k=7 d=. rc=0
```

```
k=8 d=. rc=160038
k=9 d=. rc=0
NOTE: DATA statement used (Total process time):
      real time           0.13 seconds
      cpu time            0.03 seconds

```

One more "method" that was already briefly mentioned at the beginning is `.NUM_ITEMS`, it has no arguments, you even do not use parenthesis when calling it. To be clear, documentation official calls it an *attribute*, so I will follow. The `.NUM_ITEMS` attribute returns number of items in the hash object. What does it mean? Let's look at the example below.

code: num_items

```
data have;
  do k = 1,2,2,3,3,3;
    output;
  end;
run;

data _null_;
  retain k 0;
  declare hash H1(dataset:"have");
  H1.defineKey("k");
  H1.defineDone();

  declare hash H2(dataset:"have", multidata:"y");
  H2.defineKey("k");
  H2.defineDone();

  rc1 = h1.num_items;
  rc2 = h2.num_items;
  put rc1= rc2=;
run;
```

In the log you can see the following.

the log - num_items

```
...
17     rc1 = h1.num_items;
18     rc2 = h2.num_items;
19     put rc1= rc2=;
20   run;

NOTE: There were 6 observations read from the data set WORK.HAVE.
NOTE: There were 6 observations read from the data set WORK.HAVE.
rc1=3 rc2=6
NOTE: DATA statement used (Total process time):
      real time           0.08 seconds
      cpu time            0.03 seconds
```

As you can see, both hash tables read 6 observations, but `rc1` is 3 and `rc2` is 6. The "number of items" means to the number of data entries ("observations") in the hash object. In case `H1` there were 3 keys and 3 data portions, for `H2`, the `multidata:` label forced duplicated record, so though there were only 3 unique keys, the number of elements was 6.

In the next learning step, I would like you to meet three methods that are dedicated to "waste disposal". Those methods are `.DELETE()`, `.CLEAR()`, and `.REMOVE()`. Those three methods work in a "top to

bottom" fashion. The `.DELETE()` is design to destroy whole hash object (data and metadata) so only the pointer to the hash table stays, and a brand new object can be created from scratch. Just like in this snippet.

code: delete

```
data _null_;
  declare hash H(); /* first new hash object */
  h.defineKey("k1");
  h.defineDone();

  do k1=1 to 9;
    rc = h.add();
  end;
  H.output(dataset:"K1");

  H.delete(); /* delete whole object */

  H = _NEW_ hash(); /* second new hash object */
  h.defineKey("k2","k3");
  h.defineDone();

  do k2="A","B","C";
    k3=lowcase(K2);
    rc = h.add();
  end;
  H.output(dataset:"K2");
run;
```

the log - delete

```
...
21     H.output(dataset:"K2");
22   run;

NOTE: The data set WORK.K1 has 9 observations and 1 variables.
NOTE: The data set WORK.K2 has 3 observations and 2 variables.
NOTE: DATA statement used (Total process time):
      real time           0.12 seconds
      cpu time            0.04 seconds
```

After deletion a whole new hash structure can be created from scratch.

A middle step in the "top to bottom" ladder is the `.CLEAR()` method that wipes out all data in the hash object leaving only the structure defined during declaration. You can populate hash object with data, clear it, and re-populate it with new portion of data, similarly to the example below.

code: clear

```
data _null_;
  declare hash H(ordered:"A");
  h.defineKey("k");
  h.defineDone();

  do k = "A", "B", "C";
    rc = h.add();
  end;
  H.output(dataset:"H3"); /* 3 observations */

  H.clear(); /* clear data from object */
  H.output(dataset:"H0"); /* 0 observations */
```

```
13
14   do k = "W","X","Y","Z";
15     rc = h.add();
16   end;
17   H.output(dataset:"H4"); /* 4 observations */
18 run;
```

the log - clear

```
17     H.output(dataset:"H4"); /* 4 observations */
18   run;
3
4  NOTE: The data set WORK.H3 has 3 observations and 1 variables.
5  NOTE: The data set WORK.H0 has 0 observations and 1 variables.
6  NOTE: The data set WORK.H4 has 4 observations and 1 variables.
7  NOTE: DATA statement used (Total process time):
8        real time           0.07 seconds
9        cpu time            0.04 seconds
```

The "bottom" part is the key level data deletion that is done with the `.REMOVE()` method. By pointing a particular key, that key and all its data portion is removed from the hash object. Just like in the program below.

code: remove

```
1  data _null_;
2    declare hash H();
3    h.defineKey("k");
4    h.defineDone();
5
6    do k = "A", "B", "C";
7      rc = h.add();
8    end;
9
10   rc = h.check(key:"B");
11   put "Before: " rc= ; /* success */
12
13   h.remove(key:"B"); /* clear data for a given key */
14
15   rc = h.check(key:"B");
16   put "After: " rc= ; /* fail */
17 run;
```

the log - remove

```
16     put "After: " rc= ; /* fail */
17   run;
3
4  Before: rc=0
5  After: rc=160038
6  NOTE: DATA statement used (Total process time):
7        real time           0.09 seconds
8        cpu time            0.01 seconds
```

I can imagine you saying: "Interesting, but hey, did you've just mention all data is deleted, what will happen when a key has multiple data portions?" Yes, all data is deleted. In case a key has several data portions all of them are deleted. Fortunately, there is a remedy. In situations when you need to delete only one of multiple data entries for a single key you can use the `.REMOVEDUP()` method.

Please forgive me for not presenting more details about the `.REMOVEDUP()` method in this article. This article presumes to be an informal introduction text and its volume (54 pages) is slowly starting to looking "book-ish". A highly recommended source of knowledge that covers the subject and analyses multiple data removal scenarios is the [**P. Dorfmand & D. Henderson 2018**].

Another pair of methods that are particularly useful are the `.REF()` and the `.REPLACE()` methods. They both are somehow related to the `.ADD()` method.

The first one, `.REF()`, is a syntactic-sugar that combines `.CHECK()` and `.ADD()` methods. It works the following way: first it checks if a given key exists in a hash table, and if it does not, then it adds it (with data portion), otherwise it does not do anything, hash table stays unmodified. Documentation states that the `.REF()` method is simply equivalent to

code: ref

```
1  if h.check() then h.add();
```

If key does not exist (`.CHECK()` returns non-zero) the method adds it to the hash table.

The second one, `.REPLACE()`, behaves the following way, if a given key exists in the hash table it replace *all* of its data portion with new value, otherwise just adds new entry to the hash table. Similarly to `.REMOVE()` method, the `.REPLACE()` also uses a brute force approach and replaces *all* data portion. Of course, in the "just a dictionary" approach it will not have an impact on your processing, but if you have multiple data portions such behavior is a showstopper. Fortunately, there is a workaround, the `.REPLACEDUP()` method. But as I have referred to you the [**P. Dorfmand & D. Henderson 2018**] book about the `.REMOVEDUP()` method, I will do it again for the `.REPLACEDUP()`.

HOW TO NAVIGATE OVER HASH TABLE?

From their natural construction, temporary arrays are very easy to traverse and iterate over. All cells are adjacent blocks in the memory, and you can just loop over them. Hash tables, on the other hand, work differently. First of all, hash tables were design for "ultra-fast" search and data retrieval when you provide a key, and not necessary for looping over them in situations where there is no particular key to look for. As I earlier encouraged you to think about a hash table as a bag for data, a bag does not have any predefined order. I beg your pardon for a stereotypical cliche but imagine a girl's purse. To see what is inside you just have to pick things from it one after another, *iterating* from the *first*, through the *next*, until the *last* one. There are two aspects of traversing through hash objects. The first is about moving through all "observation" in the hash object, the second about traversing data in a "multidata" key. In this section I will describe methods dedicated to those two separate cases.

Moving through hash objects - Things inside a purse will not take themselves out from the purse, they need someone who will, one-by-one, take them out. An "iterator", you might say. In case of a hash table it also needs a helper to iterate over and get data portions one after another. The helper is called **hash iterator**. To use iterator, it first has to be declared. Iterator's declaration syntax is very similar to hash table declaration and look the following

```
declare hiter iteratorName("hashTableName");
```

The declaration of the iterator (`hiter`) has to follow after declaration of hash object. In other words, in your program the `declare hiter I("H");` code hash to be written after `declare hash H(...);`. Two following declaration will work equivalently.

code: hiter

```
1  data _null_;
2    k=17;
3    declare hash H();
```

```
  declare hiter I("H");
  h.defineKey("k");
  h.defineDone();
run;

data _null_;
  k=42;
  declare hash H();
  h.defineKey("k");
  h.defineDone();
  declare hiter I("H");
run;
```

Personally, I prefer the second one, but only because SAS syntax highlighting gives me clear view of the beginning and the end of the declaration, nothing more. Of course, a declaration with use of the `_NEW_` statement works too.

Assuming that the purse has now a person that will take its content out, let's learn how to iterate. After executing the snipped below the memory state just before the end of the second DATA step looks the following way.

code

```
data have;
  k=1; d="A"; output;
  k=2; d="B"; output;
  k=3; d="C"; output;
run;
data _null_;
  if 0 then set have;
  declare hash H(dataset:"have");
  h.defineKey("k");
  h.defineData("d");
  h.defineDone();
  declare hiter I("H");
stop;
run;
```

Program Data Vector

k [num,8]	d [char,1]	_N_	_ERROR_
.		1	0

H [hash]

k [num,8]	d [char,1]
2	B
1	A
3	C

◀ I

The only difference in the memory layout is the "◀ I" symbol. It is a symbolic representation of a data pointer that the iterator `I` use to point particular "observation" within the hash table. At the beginning the iterator does not point to any data portion in the hash table. To start iterating over the data portion of hash table you have to locate the pointer at the *first* or the *last* "observation" and then advance to the *next* or the *previous* element respectively. Methods, this time iterator's methods, which help to do the job are: `.FIRST()`, `.LAST()`, `.NEXT()`, and `.PREV()`. In the moment the iterator points to an "observation" in the hash table, all data portion is copied into PDV. The following snippet uses the `.FIRST()` method to start traversing over the hash table, and then the `.NEXT()` method is called three time.

code: first & next

```
data have;
  k=1; d="A"; output;
  k=2; d="B"; output;
```

```
  k=3; d="C"; output;
run;
data _null_;
  if 0 then set have;
  declare hash H(dataset:"have");
  h.defineKey("k");
  h.defineData("d");
  h.defineDone();
  declare hiter I("H");

  call missing(k,d);
  put "0) " _all_;
  rc=I.first();
  put "1) " _all_;
  rc=I.next();
  put "2) " _all_;
  rc=I.next();
  put "3) " _all_;
  rc=I.next();
  put "4) " _all_;
stop;
run;
```

the log - first & next

```
...
22     rc=I.next();
23     put "4) " _all_;
24   stop;
25   run;

NOTE: There were 3 observations read from the data set WORK.HAVE.
0) k=. d=  rc=. _ERROR_=0 _N_=1
1) k=. d=B rc=0 _ERROR_=0 _N_=1
2) k=. d=A rc=0 _ERROR_=0 _N_=1
3) k=. d=C rc=0 _ERROR_=0 _N_=1
4) k=. d=C rc=160038 _ERROR_=0 _N_=1
NOTE: DATA statement used (Total process time):
      real time           0.21 seconds
      cpu time            0.12 seconds
```

Notice how only the data portion, i.e., the variable d, is updated in the PDV. Observe that the third call to the `.NEXT()` method returns non-zero, what means there is no next element. In the traversing process the `.FIRST()` method locates pointer on the first element and the data is transferred to PDV.

Program Data Vector

k [num,8]	d [char,1]	rc [num,1]	_N_	_ERROR_
.	B	0	1	0

H [hash]

k [num,8]	d [char,1]
2	B
1	A
3	C

◀ I

First call to the `.NEXT()` moves pointer to the second position, the d is updated with new value.

Program Data Vector				
k [num,8]	d [char,1]	rc [num,1]	_N_	_ERROR_
.	A	0	1	0

H [hash]	
k [num,8]	d [char,1]
2	B
1	A
3	C

◀ I

Second call to the `.NEXT()` moves pointer to the third, the last, position in the hash table, the d is updated with new value to "C".

Program Data Vector				
k [num,8]	d [char,1]	rc [num,1]	_N_	_ERROR_
.	C	0	1	0

H [hash]	
k [num,8]	d [char,1]
2	B
1	A
3	C

◀ I

Third call to the `.NEXT()` returns 160038, what means "there is no next element", pushes the pointer out of the hash object, ant value of d is not changed.

Program Data Vector				
k [num,8]	d [char,1]	rc [num,1]	_N_	_ERROR_
.	C	160038	1	0

H [hash]	
k [num,8]	d [char,1]
2	B
1	A
3	C

◀ I

In this example the number of elements was known, whence the code snippet could have just four calls to traversing methods. In practical application a loop seems to be more practical approach.

code: next in a loop

```
1  data have;
2    k1=1; k2="A"; output;
3    k1=2; k2="B"; output;
4    k1=3; k2="C"; output;
5  run;
6  data _null_;
7    if 0 then set have;
8    declare hash H(dataset:"have");
9    h.defineKey("k1");
10   h.defineKey("k2");
11   h.defineDone();
12   declare hiter I("H");
13
14   do j = 1 by 1 while(I.next()=0);
15     put j= k1= k2=;
16   end;
17 stop;
18 run;
```

the log - next in a loop

```
...
16      end;
17    stop;
18    run;
```

```
5
6  NOTE: There were 3 observations read from the data set WORK.HAVE.
7  j=1 k1=1 k2=A
8  j=2 k1=2 k2=B
9  j=3 k1=3 k2=C
10 NOTE: DATA statement used (Total process time):
11       real time           0.11 seconds
12       cpu time            0.07 seconds
```

There is only one "gotcha ya"! To write a loop with *only* the `.NEXT()` method the pointer has to be located outside data portion before the loop starts. If the pointer is located outside the hash table data portion, then the first call to `.NEXT()` behaves like a call to the `.FIRST()` method and sets the pointer at the first position. In case the analyzed example the situation allowed for simple `DO-WHILE` loop. But in case your iterator points inside the data portion, somehow you have to force the pointer outside the hash. Fortunately, a very simple programming trick does the job. All that you need to run before the loop is:

```
rc=I.last();rc=I.next();
```

those two instruction simply move the pointer at the end of the data list and push it of it. Equivalently

```
rc=I.first();rc=I.prev();
```

can be used. The loop from the snippet above can be easily modified to iterate over the hash table backward

code: prev in a loop

```
13 ...
14   do j = 1 by 1 while(I.prev()=0);
15     put j= k1= k2=;
16   end;
17 stop;
18 run;
```

One more method requires a bit of attention. The `.SETCUR()` method is a convenient one. It allows the start iterator's route not on the first or the last element of the hash table but at the position indicated by any key provided to executed `.SETCUR()` method.

code: setcur

```
1  data _null_;
2    declare hash H(ordered:"Descending");
3    h.defineKey("k");
4    h.defineDone();
5    declare hiter I("H");
6
7    do k = 1, 2, 3, 4, 5, 6;
8      rc=h.add();
9    end;
10
11   j=i.setcur(key:3);
12   put j= k= ;
13   do j = 1 by 1 while(i.next()=0);
14     put j= k= ;
15   end;
16 stop;
17 run;
```

the log - setcur

```
...
15      end;
16    stop;
17    run;

j=0 k=3
j=1 k=2
j=2 k=1
NOTE: DATA statement used (Total process time):
      real time           0.05 seconds
      cpu time            0.04 seconds

```

Traversing data for a "multidata" key - The process of data extraction where you just want to loop over every data portion "from top to bottom" and get those data is very simple and can be done in two ways. The first approach uses well known `.FIND()` method and additionally its counterpart the `.FIND_NEXT()` method. The DATA step below is an example of how this solution works. The conditionally executed `.FIND()` method checks if a given key exists at all in the hash table. If yes, then the data portion is loaded to the PDV, as the method usually does. After entering the body of the loop code does what it is asked to do (here, prints values in the LOG) and at the bottom of the `DO-UNTIL` loop the condition runs the `.FIND_NEXT()` method to verify if that key group has next element. If yes, the loop iterates again, if no, the loop terminates. I want to remind you about one more thing: explicit use of the `key:` label gives one more observation, the `k` variable is not data portion variable and is not updated!

code: find_next

```
data have;
  k=1; d="C"; output;
  k=1; d="B"; output;
  k=1; d="A"; output;
  k=2; d="X"; output;
run;
data _null_;
  if 0 then set have;
  declare hash H(dataset:"have", multidata:"Y");
  h.defineKey("k");
  h.defineData("d");
  h.defineDone();

  if 0=H.find(key:1) then do j = 1 by 1 until(H.find_next(key:1));
    put j= k= d=;
  end;
stop;
run;
```

In the LOG you can see that all values for key 1 are printed, and, as observed earlier, in the exact order they were read in into the hash table.

the log

```
...
18    run;

NOTE: There were 4 observations read from the data set WORK.HAVE.
j=1 k=. d=C
j=2 k=. d=B
j=3 k=. d=A
NOTE: DATA statement used (Total process time):
```

```
      real time              0.04 seconds
      cpu time               0.04 seconds
```

The second approach involves the `.DO_OVER()` method (do not be confused, this is different "do over" than the array looping "do over" or CAS-L language one). The `.DO_OVER()` method takes key and locates it in the hash table The verification is done first, that is why the `DO-WHILE` loop is used. If the data portion exists the method returns zero and extracts the data portion into the PDV.

code: do_over

```
data _null_;
  if 0 then set have;
  declare hash H(dataset:"have", multidata:"Y");
  h.defineKey("k");
  h.defineData("d");
  h.defineDone();

  do while(0=H.do_over(key:1));
    put k= d=;
  end;
stop;
run;
```

Result, printed in the LOG, is exactly the same as in the previous approach. Also, the same remark about explicit use the `key:` label is observable here.

As I mentioned at the beginning of this sub-section, the "simple" iteration over data portion block for a given key is indeed simple. If your target is to execute more fancy looping, e.g., a backward loop, you have to put some more effort. In particular, you will probably use combination of the `.FIND()` and `.FIND_NEXT()`/`.FIND_PREV()` methods in combination with two additional methods, namely, the `.HAS_NEXT()` and the `.HAS_PREV()` method. Pardon me again that I only signalize this topic and not dive deeper into it, but recommended earlier, the [**P. Dorfmand & D. Henderson 2018**] explains all the intricacies of the process in every detail.

There is one more method that I want to mention in context of how the `.DO_OVER()` method works. As long as the iteration goes over the whole data elements you may not even notice the issue. Let's look at the following program, the `DO-LOOP` changes the key value from 1 to 2 and back and forth, the `.DO_OVER()` method is called inside the loop body, but the printed out values are nothing like expected... why the hell, though the key is changing, data seems to be all the time taken from the "1" group? Well, no "seems to be", they are from group "1". This is the `.DO_OVER()` method's behavior that needs an explanation.

code

```
data have;
  length k 8 d $ 2;
  do k = 1,2;
    do d="a","b","c";
      d=cats(d,k);
      output;
    end;
  end;
run;
data _null_;
  if 0 then set have;
  declare hash H(dataset:"have", multidata:"Y", ordered:"a");
  h.defineKey("k");
  h.defineData("d");
  h.defineDone();
```

```
16   do k = 1, 2, 1, 2, 1, 2;
17     rc=h.do_over(key:k);
18     put k= d= rc=;
19   end;
20 stop;
21 run;
```

After the code executes

the log

```
1  ...
2  17     do k = 1, 2, 1, 2, 1, 2;
3  18       rc=h.do_over(key:k);
4  19       put k= d= rc=;
5  20     end;
6  21   stop;
7  22   run;
8
9  NOTE: There were 6 observations read from the data set WORK.HAVE.
10 k=1 d=a1 rc=0
11 k=2 d=b1 rc=0
12 k=1 d=c1 rc=0
13 k=2 d=c1 rc=160038
14 k=1 d=a1 rc=0
15 k=2 d=b1 rc=0
16 NOTE: DATA statement used (Total process time):
17       real time           0.05 seconds
18       cpu time            0.04 seconds
```

Before the loop in line 17 starts execution the memory layout is like in the figure below, PDV has no data yet, hash table is populated with the data, and the `.DO_OVER()` method pointer "◀ DO" points outside key groups.

Program Data Vector

k [num,8]	d [char,2]	rc [num,1]	_N_	_ERROR_
.		.	1	0

H [hash]

k [num,8]	d [char,2]
1	A1
1	B1
1	C1
2	A2
2	B2
2	C2

◀ DO

When the first iteration starts variable k became 1, then the `.DO_OVER()` method is executed. The method picks up a key from PDV, search it in the hash table, and (since found) fixes the pointer at the beginning of this key group, and updates PDV with data portion taken from the hash table, just like in the figure below.

Program Data Vector

k [num,8]	d [char,2]	rc [num,1]	_N_	_ERROR_
1	A1	0	1	0

H [hash]

k [num,8]	d [char,2]
1	A1
1	B1
1	C1
2	A2
2	B2
2	C2

◀ DO(1)

When the second iteration starts variable k became 2, then the .DO_OVER() method is executed, but the method *does not* picks up key from PDV! The .DO_OVER() keeps fixed value of the key until it gets to the end of that key's group. The method only moves the pointer to the next position of this key group, and updates PDV with data portion taken from the hash table, just like in the figure below.

Program Data Vector

k [num,8]	d [char,2]	rc [num,1]	_N_	_ERROR_
2	B1	0	1	0

H [hash]

k [num,8]	d [char,2]
1	A1
1	B1
1	C1
2	A2
2	B2
2	C2

◀ DO(1)

Iteration three does not make any difference, except that k became 1 again, pointer is shifted to next position, data are updated in the PDV.

Program Data Vector

k [num,8]	d [char,2]	rc [num,1]	_N_	_ERROR_
1	C1	0	1	0

H [hash]

k [num,8]	d [char,2]
1	A1
1	B1
1	C1
2	A2
2	B2
2	C2

◀ DO(1)

Iteration number four changes key value to 2 again, the .DO_OVER() executes and this time attempt to move the pointer to the next data element fails because there is no data, the method returns 160038 and the pointer is pushed outside the group. Variable d is *not* updated.

Program Data Vector

k [num,8]	d [char,2]	rc [num,1]	_N_	_ERROR_
2	C1	160038	1	0

H [hash]

k [num,8]	d [char,2]
1	A1
1	B1
1	C1
2	A2
2	B2
2	C2

◀ DO(1)

Only when the pointer is pushed of the group of the key the .DO_OVER() method can change pointed group. Unfortunately, in the fifth iteration variable k holds 1 again, so the .DO_OVER() once more looks for 1 in the hash table, sets the pointer at the beginning of the group and extracts "A1" to the PDV.

Comparing to the length of the example presenting the issue and the explanation of it, the resolution is surprisingly short. To avoid situations like the one above you have to use the .RESET_DUP() method. When the pointer set by the .DO_OVER() is somewhere in the middle of a key's group and you want to change the key and start fresh iteration you execute the .RESET_DUP() method before the .DO_OVER(). DATA step code snippet below resets the pointer after every second iteration, this way for every key only two first data portions are extracted.

code: reset_dup

```
1 data _null_;
2   if 0 then set have;
```

```
  declare hash H(dataset:"have", multidata:"Y", ordered:"A");
  h.defineKey("k");
  h.defineData("d");
  h.defineDone();

  do k = 1, 2;
    do i="frst", "scnd";
    rc=h.do_over(key:k);
    put i= k= d= rc=;
    end;
    rc=h.reset_dup();
  end;
stop;
run;
```

the log - reset_dup

```
...
13        rc=h.reset_dup();
14      end;
15    stop;
16    run;

NOTE: There were 6 observations read from the data set WORK.HAVE.
i=frst k=1 d=a1 rc=0
i=scnd k=1 d=b1 rc=0
i=frst k=2 d=a2 rc=0
i=scnd k=2 d=b2 rc=0
NOTE: DATA statement used (Total process time):
      real time           0.05 seconds
      cpu time            0.03 seconds
```

THE LIST

Since this is a tutorial style article it cannot stand without the list of statements, methods and attributes associated with hash tables. In fact, the list is direct alphabetic extract from the documentation (as of October 12th, 2026 for SAS 9.4M9). Up to now a lot of those listed below were discussed but not all or not in the depth they should, that is why the list is marked with ✔ and ✘ symbols to indicate need for future reading.

- ADD Method [✔]
- CHECK Method [✔]
- CLEAR Method [✔]
- DECLARE Statement: Hash [✔/✘[4]] and Hash Iterator Objects [✔]
- DEFINEDATA Method [✔]
- DEFINEDONE Method [✔]
- DEFINEKEY Method [✔]
- DELETE Method: Hash [✔] and Hash Iterator Objects [✔]
- DO_OVER Method [✔]
- EQUALS Method [✘]
- FIND Method [✔]
- FIND_NEXT Method [✔]
- FIND_PREV Method [✔]

[4] Not all labels were discussed.

- FIRST Method [✔]
- HAS_NEXT Method [✘]
- HAS_PREV Method [✘]
- ITEM_SIZE Attribute [✔]
- LAST Method [✔]
- Hash and Hash Iterator Operator: Objects (_NEW_) [✔]
- NEXT Method [✔]
- NUM_ITEMS Attribute [✔]
- OUTPUT Method [✔]
- PREV Method [✔]
- REF Method [✔]
- REMOVE Method [✔]
- REMOVEDUP Method [✘]
- REPLACE Method [✔]
- REPLACEDUP Method [✘]
- RESET_DUP Method [✔]
- SETCUR Method [✔]
- SUM Method [✘]
- SUMDUP Method [✘]

EXECUTION TIME ANIMAL

In one of the first sessions above, when I told you about how a hash table is declared and initialized I have mentioned that hash tables are initialized and instantiated in the executions phase. Such behavior has important consequences. In every example, up to now, DATA steps had only one iteration of the implicit loop. It was so because either there was no data reading statement (e.g., `SET`) in the code that would triggered the loop or there was an explicit `STOP` statement that was preventing the second iteration.

Let's look at a situation where you would like to create a simple report table. Your first step is a simple look up process over a dictionary (i.e., hash table) and regularly "`SET`ed" (sequentially read) data. Let's imagine some company employees data base with data looking like this

code: employees data stets

```
data employees;
  length id 8 dept $ 1;
  do id = 1 to 9 by 2, 0 to 8 by 2; /* 10 obs. */
    d+1;
    dept=choosec(mod(d,4)+1,"S","A","H","S");
    output;
  end;
  drop d;
run;
```

code: departments data stets

```
data departments;
  length dept $ 1 desc $ 32;
  dept = "A"; desc="Applications R & D"; output;
  dept = "S"; desc="Sales & Marketing";  output;
  dept = "H"; desc="Human Resources";    output;
run;
```

You may think code like below will give you the result, and you will not be wrong, the `report1` data set will contain expected data.

code: report1 data set

```
data report1;
  declare hash D(dataset:"departments");
  D.defineKey("dept");
  D.defineData("desc");
  D.defineDone();

  set employees;

  length desc $ 32;
  if D.find() then call missing(desc);

  output;
run;
proc print data=report1;
run;
```

Printed table looks like the following and nothing seems to be a problem...

Obs	id	dept	desc
1	1	A	Applications R & D
2	3	H	Human Resources
3	5	S	Sales & Marketing
4	7	S	Sales & Marketing
5	9	A	Applications R & D
6	0	H	Human Resources
7	2	S	Sales & Marketing
8	4	S	Sales & Marketing
9	6	A	Applications R & D
10	8	H	Human Resources

Well, if you are a SAS user (i.e., only look at the output window) you will not even notice a critical flaw the program has... but if you are a SAS programmer (i.e., you always inspect the LOG first) you instantly see there is something off... The LOG looks like the following.

the log - report1 data set

```
9       length desc $ 32;
10      if D.find() then call missing(desc);
11
12      output;
13    run;

NOTE: There were 3 observations read from the data set WORK.DEPARTMENTS.
NOTE: There were 3 observations read from the data set WORK.DEPARTMENTS.
NOTE: There were 3 observations read from the data set WORK.DEPARTMENTS.
NOTE: There were 3 observations read from the data set WORK.DEPARTMENTS.
NOTE: There were 3 observations read from the data set WORK.DEPARTMENTS.
NOTE: There were 3 observations read from the data set WORK.DEPARTMENTS.
NOTE: There were 3 observations read from the data set WORK.DEPARTMENTS.
NOTE: There were 3 observations read from the data set WORK.DEPARTMENTS.
NOTE: There were 3 observations read from the data set WORK.DEPARTMENTS.
NOTE: There were 3 observations read from the data set WORK.DEPARTMENTS.
NOTE: There were 3 observations read from the data set WORK.DEPARTMENTS.
NOTE: There were 10 observations read from the data set WORK.EMPLOYEES.
```

```
NOTE: The data set WORK.REPORT1 has 10 observations and 3 variables.
NOTE: DATA statement used (Total process time):
      real time           0.01 seconds
      cpu time            0.01 seconds
```

The LOG shows the dictionary data set was read 11(!) times. That is exactly what the "hash table is an execution phase animal" sentence means. Whenever the DATA step's implicit loop iterates, those parenthesis in the declare statement:

```
declare hash D(dataset:"departments");
```

instruct SAS to create a new instance of hash table `H` and read data from the `WORK.DEPARTMENTS` into it. Whence `H` is created one time and recreated 10 times. One more important thing has to be *emphasized*, when a new hash table is instantiated the old one is not destroyed, the pointer indicating its location in memory is replaced with a new one, the old one is only "detached" from contact with PDV and stays in the memory until the DATA step end. You will soon see that, when properly monitored, those "in memory leftovers" can be used for your advantage, but for now they are only resource wasting garbage.

The simplest solution to prevent SAS from recreating the hash table the is to make the hash table code only one time. If the DATA step has only one iteration then issue is trivially solved. In case your DATA step will be looping, the solution is to enclose hash table declaration/instantiation in conditional statement, for example like this:

```
if 1 = _N_ then do; ...  end;
```

This way of fixing is used in probably 99% of programs using hash tables. The above example code, modified to fix the problem, looks like the following

code: report1 fixed

```
data report1;
  if 1 = _N_ then
    do;
      if 0 then set employees departments;
      declare hash D(dataset:"departments");
      D.defineKey('dept');
      D.defineData('desc');
      D.defineDone();
    end;

  set employees;

  if D.find() then call missing(desc);

  output;
run;
```

The result is exactly the same, but now log looks definitely better this time.

the log

```
13     if D.find() then call missing(desc);
14
15     output;
16   run;

NOTE: There were 3 observations read from the data set WORK.DEPARTMENTS.
NOTE: There were 10 observations read from the data set WORK.EMPLOYEES.
NOTE: The data set WORK.REPORT1 has 10 observations and 3 variables.
NOTE: DATA statement used (Total process time):
```

```
10       real time              0.06 seconds
11       cpu time               0.04 seconds
```

Additional "`if 0 then set ...;`" code in line number 4 is added to get metadata "for free" and, in particular, to keep the order of variables.

ONE OR TWO EXAMPLES

In this section I would like to present you with a few example where hash tables are used.

Aggregation - The first example will use data from previous (slightly modified) section and will be a simple aggregation.

code: employees data stets

```
1  data employees;
2    length id 8 dept $ 1;
3    do id = 1 to 9 by 2, 0 to 8 by 2; /* 10 obs. */
4      d+1;
5      dept=choosec(mod(d,4)+1,"S","A","H","S");
6      output;
7    end;
8    id=11; dept="U"; /* obs. 11th */
9    output;
10   drop d;
11 run;
```

code: departments data stets

```
1  data departments;
2    length dept $ 1 desc $ 32;
3    dept = "A"; desc="Applications R & D"; output;
4    dept = "S"; desc="Sales & Marketing";  output;
5    dept = "H"; desc="Human Resources";    output;
6    dept = "E"; desc="Trainings & Edu";    output;
7  run;
```

The task is: count the number of employees working in every department and present a report with full departments names, for simplicity in the log; make it smart, i.e., handle empty departments and entries not covered by the dictionary.

This sounds like a job for several DATA and PROC steps, but with help of two hash tables it all can be done in one organized data step. Here is the code. It is composed of five parts, two of them prepare structure of hash tables, remaining three execute the process, i.e. aggregation, reporting, exceptions handling. Before detailed explanation look through the code yourself, with what you have read up to now you should be able to grasp the idea.

code: employees report

```
1  data _null_;
2    /* 1) create dictionary of departments names */
3    if 0 then set departments;
4    declare hash D(dataset:"departments",ordered:"A");
5    D.defineKey("dept");
6    D.defineKey("desc");
7    D.defineDone();
8    declare hiter I("D");
9
```

```
  /* 2) prepare table for aggregation */
  declare hash A();
  A.defineKey("dept");
  A.defineData("dept", "n");
  A.defineDone();

  /* 3) do the aggregation */
  do until (_E_);
    set employees end=_E_;

    if A.find() then n=1;
                else n+1;
    A.replace();
  end;

  /* 4) print report to the output */
  do while(0 = I.next());
    if A.find() then n=0;

    put "Department: " desc $quote. "has " n "employees.";
    rc = A.remove();
  end;

  /* 5) handle "surprises" */
  if A.num_items then
    do;
      put / "WARNING: Unknown department found!";
      A.output(dataset:"Unknown_department");
    end;
stop;
run;
```

the log

```
39   stop;
40   run;

NOTE: There were 4 observations read from the data set WORK.DEPARTMENTS.
Department: "Applications R & D"              has 3 employees.
Department: "Trainings & Edu"                 has 0 employees.
Department: "Human Resources"                 has 3 employees.
Department: "Sales & Marketing"               has 4 employees.

WARNING: Unknown department found!
NOTE: The data set WORK.UNKNOWN_DEPARTMENT has 1 observations and 2 variables.
NOTE: There were 11 observations read from the data set WORK.EMPLOYEES.
NOTE: DATA statement used (Total process time):
      real time           0.17 seconds
      cpu time            0.06 seconds
```

The LOG looks good, problems are handled correctly, now let's answer the big question: *What does this DATA step do?* From the top:

(1) Conditional `SET` statement covers the metadata for the dictionary. Hash table `D` reads the `departments` data set, variables `dept` and `desc` define a composed key. Iterator `I` is declared to be later used in the reporting part of the process.

(2) An empty hash table `A` is declared. Variable `dept` is defined as key and data, and variable `n` is defined as data portion storing employees count.

(3) The DO-UNTIL loop iterates over every observation of the employees data set until it runs out of data (_E_ becomes 1 at the last observation). After observation is read, the .FIND() method verifies if dept variable value in PDV exists in the hash table A. If not (method returns a not zero value) the n variable is initially set to 1, othervise n is incremented by 1. At the bottom of the loop, the .REPLACE() method adds new key or updates existing one.

(4) The DO-WHILE loop iterates over the dictionary hash table D, the driver is the I iterator for which the .NEXT() method updates values in the PDV. After every update aggregating hash table A is checked for key value. If check is successful the .FIND() method extracts accumulator n, otherwise n is set to zero. Report is printed. At the bottom of the loop current key and its data portion are removed from the hash table A.

(5) The last step is conditionally executed only if the hash table A still has elements, i.e. there is dept value that was not found in the dictionary. If this is the case, a warning is produced and the .OUTPUT() method produces a data set that contains code and count of problematic departments.

I hope you have noticed there was no other than in-memory ordered:"A" data sorting involved in the process. This is yet another advantage of using hash tables, oftentimes one of the most costly operations, i.e., data sorting, can be totally avoided. Next example highlights this even more.

Multi-variables data joining without data sorting - Next example shows how hash tables can optimize input/output performance. The example is taken almost verbatim form the [**B. Jabłoński, PharmaSUG 2025**] article. The "base" data set is smaller and has only 1150 observations and 4 variables.

code: source.have

```
data source.have;
  call streaminit(42);
  do grp = 1,3,5, 2,4,6;
    do id = "A","X","B","C","Y","Z";
      do number = 0 to rand("integer",17,42);
        obs+1;
        output;
      end;
    end;
  end;
run;
```

The situation is the case where you have to add (or combine) some new data into the base table, furthermore:

(1) the data are provided in separate data sets (no Excels, CSVs, etc., just for simplicity),

(2) the data you want to combine must be merged over different variables, and

(3) the final data set should maintain the original observations order.

For simplicity, three data sets with additional data have only two variables each, one variable for "joining" and the other for data that needs to be added, e.g., grp and grp_text, with values 1,2,3... and "group 1","group 2","group 3",... I know it is a school-like looking data, but it is more than enough to explain the idea.

Usually, two approaches are used to solve the task. The first one combines merging and sorting intertwined together, the second approach, as one can expect, is SQL's left joining.

The "merge and sort" solution starts with ordering all additional data sets by their merging variables. Next, the base data set is sorted by selected variable and then merged with additional data. The process is repeated for all variables, and eventually the final data set is resorted to restore the original observations order. In the example the process involves 23I/O (1I/O means either one data read or one data write statement, regardless of data set size). The alternative, with PROC SQL, seems to be less I/O intense, at least at the first glance. Unfortunately, the _METHOD option reveals it is not just a '94 "Four Reads and a Write" romantic comedy but it is much more complicated and more input/output extensive. The LOG shows that before joining,

all data sets are resorted, and since the merge join is used, the temporary data have to be stored in utility files. Of course the final re-sorting takes its toll too.

code: Merge and sort

```
proc sort data = number_data
  out = number_data_sort;
  by number;
run;
proc sort data = id_data
  out = id_data_sort;
  by id;
run;
proc sort data = grp_data
  out = grp_data_sort;
  by grp;
run;
proc sort data = source.have
  out = have_by_grp;
  by grp;
run;
data have_1;
  merge have_by_grp
        grp_data_sort;
  by grp;
run;
proc sort data = have_1
  out = have_by_id;
  by id;
run;
data have_2;
  merge have_by_id
        id_data_sort;
  by id;
run;
proc sort data = have_2
  out = have_by_number;
  by number;
run;
data have_3;
  merge have_by_number
        number_data_sort;
  by number;
run;
proc sort data = have_3
  out = combined_DS;
  by obs;
run;
```

code: SQL

```
proc sql _method;
  create table combined_SQL as
  select
    h.*,
    n.number_text,
    i.id_text,
    g.grp_text
  from
    source.have as h
  left join
    number_data as n
    on h.number=n.number

  left join
    id_data as i
    on h.id=i.id

  left join
    grp_data as g
    on h.grp=g.grp

  order by obs;
quit;
```

The LOG unveils it all. Even if we assume that the `sqxsrc` and `sqxsort` utilize `I/O` operations together we can count something around eleven `I/O` in the process.

the log - SQL and _method

```
NOTE: SQL execution methods chosen are:
      sqxcrta
          sqxsort
              sqxjm
                  sqxsort
                      sqxsrc( WORK.GRP_DATA(alias = G) )
```

```
7                   sqxsort
8                       sqxjm
9                           sqxsort
10                              sqxsrc( WORK.ID_DATA(alias = I) )
11                          sqxsort
12                              sqxjm /* Merge join operation */
13                                  sqxsort
14                                      sqxsrc( WORK.NUMBER_DATA(alias = N) )
15                                  sqxsort /* Sort operation */
16                                      sqxsrc( SOURCE.HAVE(alias = H) )
17 NOTE: SAS threaded sort was used.
```

This time, with a little help from our hash table friends, the "optimized" solution proposed here uses only 5I/O: three data reads to populate hash tables: H1, H2, and H3 with data, one run over the source.have base data set, and eventually for writing the final combined_hash data set, with no need for any re-sorting at all (which also reduces Operating System Memory cost by 15 times!). The I/O costs reduction goes 23I/O$\longrightarrow$$5$I/O. And just to be clear, code in line number 2 does not read any data during the execution phase. The SET statement is there just for metadata extraction in the compilation phase and is used just to minimize variables metadata interaction (simple LENGTH would work too).

code: hash tables approach

```
1  data combined_hash;
2    if 0 then set work.number_data work.id_data work.grp_data;
3
4    declare hash H1(dataset:"work.number_data");
5    H1.defineKey("number");
6    H1.defineData("number_text");
7    H1.defineDone();
8
9    declare hash H2(dataset:"work.id_data");
10   H2.defineKey("id");
11   H2.defineData("id_text");
12   H2.defineDone();
13
14   declare hash H3(dataset:"work.grp_data");
15   H3.defineKey("grp");
16   H3.defineData("grp_text");
17   H3.defineDone();
18
19   do until(_E_);
20     set source.have
21         end=_E_;
22     if H1.find() then number_text="";
23     if H2.find() then id_text="";
24     if H3.find() then grp_text="";
25     output;
26   end;
27 stop;
28 run;
```

Key jumper - This is an example of how traversing a hash tables can be done with technique I am calling a "key jumping". Imagine you have data with multiple keys, and you have to traverse them in different order than the way they would be ordered in hash table. There are multiple different ways to force reordering, the one I will present here came up to me during one of discussions at the SAS-L discussion list in October 2018. Let's say the data set is

code: have

```
data HAVE;
  input k1 k2 $ d;
cards;
1 A 1
1 B 2
1 C 11
1 D 22
2 A 3
2 B 4
2 C 33
2 D 44
3 A 5
3 B 6
3 C 55
3 C 66
4 A 7
4 B 8
4 C 77
4 E 88
;
run;
```

The data set HAVE has two key variables k1 and k2 and one data portion d. Imagine you need to organize the data in order ascending by one variable in the key and descending by another, let's say you have a macro %ABC(k1,k2,d) and you want to run it with the CALL EXECUTE subroutine in that particular order.

code: ABC

```
%macro ABC(k1,k2,d);
data _null_;
  k1 = symget("k1");
  k2 = symget("k2");
  d  = symget("d");
  put / "[key2-jumper outer loop] " k2= ;
  put @2 "[key1-jumper inner loop] " k1=;
  put @4 "[output] " (_all_) (=);
run;
%mend ABC;
```

I know you may say that this type of code execution can be done in different way, but this is just to present the technique, just for the learning purpose. You cannot know in what situations you find it useful.

The DATA step looks like the following

code: keys jumping

```
/* double key */
data _null_;
  if 0 then set HAVE;
  /* hash for the data */
  declare hash H(dataset:"HAVE(obs  = 0)", multidata:"Y");
  H.DefineKey("K1", "K2");
  H.DefineData("D");
  H.DefineDone();

  /* 2 hashes for _unique_ values of keys */
  declare hash KJ1(ordered:"A");
  KJ1.DefineKey("K1");
  KJ1.DefineDone();
  declare hiter I1("KJ1");
```

```
15
16    declare hash KJ2(ordered:"A");
17    KJ2.DefineKey("K2");
18    KJ2.DefineDone();
19    declare hiter I2("KJ2");
20
21    /* load data - read data into all three hashes at one read */
22    do until(_E_);
23        set HAVE end = _E_;
24
25        rc=H.ADD();     /* keys and (multi)data */
26        rc=KJ1.ADD();   /* only k1 values */
27        rc=KJ2.ADD();   /* only k2 values  */
28    end;
29
30    /* manipulate the order */
31    do while(0 = I2.prev()); /* looping backward */
32      do while(0 = I1.next()); /* looping forward */
33        /* loop over data portion of selected keys pairs */
34        if 0 = H.FIND() then
35          do j=1 by 1 until (H.FIND_NEXT());
36            call execute(cats('%nrstr(%ABC(',k1,",",k2,",",d,"))"));
37          end ;
38      end;
39    end;
40  stop;
41  run;
```

Those two additional hash tables role is to allow convenient swaps between key changes, kind of like a (not existing) `.NEXT_KEY()` method that could push processing to a next key after looping over whole data group of a current key. With regard to the keys ordering, because now you can organize it as you wish.

How does it work? The idea is shamefully simple. First you create hash table for your data. Next you create two (or more, if need be) auxiliary hash tables that keep only unique values of variables building the key. Adding `ordered:"A"` in both makes ordered looping trivial. The `DO-UNTIL` loop iterates over input data set and populates all three hash tables in one data read. The final part is just a double `DO-WHILE` loop forcing the order you want to have, in this case the first loop iterates backward over `k2` using the `.PREV()` method and the second loop iterates forward over `k1` using the `.NEXT()` method. Conditional loop inside those two loop is just to traverse data portion for a given key and to call the macro.

Hash of hashes - Since I have already pulled up that old cliche joke about a woman's purse... A purse can hide another purse inside it, a purse of purses... Hash tables can do that too. You can create a construction named a hash of hashes (HoH for short). The first publication I could find to refer to is the [**P. M. Dorfman, K. Vyverman, SUGI 2005**] article. Though concept of hash of hashes was introduced earlier. The idea was first brought to discussion by Richard A. DeVenezia during 2003 SESUG conference in Saint Pete Beach. SAS-L discussion and prototypes based on arrays and macro programming, like for example: `https://www.devenezia.com/downloads/sas/macros/index.php?m=multi-level-hash` were discussed before the paper.

This idea takes advantage of the fact that *hash table data portion can be used to store pointers to other hash tables*! A brilliant idea in its simplicity! Again, the example will be very simple. Imagine you have the following a data set.

code: have

```
1  data have;
2    input X Y Z $;
```

```
cards;
1 10 A
2 11 B
1 12 C
2 13 D
3 14 E
1 15 F
3 16 G
1 17 H
2 18 I
3 19 J
;
run;
```

Your job is to split `have` into smaller parts according to values of variable `X`. Names of created data sets should be `group*`, where `*` is replaced with appropriate value of `X`. The input data set is not sorted, but the result data sets should be sorted descending by variable `Y`. This sounds like a job for several DATA/PROC steps, and usually it is, but if you decide to use hash of hashes technique it all can be done in one DATA step, and not even very long one. Thirty lines only! Here is the code, explanation follows.

code: hash of hashes

```
data _null_;
  declare hash H; /* H will point hash objects to store data for particular X */

  declare hash HoH(ordered: "A");
  HoH.DefineKey("X");       /* HoH keep values of X as keys    */
  HoH.DefineData("X", "H"); /* and pointers to Hs in data part */
  HoH.DefineDone();
  declare hiter I("HoH");

  do until(_E_);
    set have end = _E_;

    if HoH.Check() then /* check if X is already in HoH, */
      do; /* if not instantiate _new_ object associated with current X */
        H = _NEW_ hash(ordered: "D", multidata: "Y");
        H.DefineKey("Y");
        H.DefineData("Y", "Z");
        H.DefineDone();
        HoH.replace(); /* add association to HoH */
      end;

    HoH.find(); /* find pointer to hash H associated with current X        */
    H.Add();    /* add data to the instance of H correlated with current X */
  end;

  do while(I.next()=0);
    H.output(dataset: cats("work.group",X));
  end;
stop;
run;
```

the log

```
29    stop;
30    run;

NOTE: The data set WORK.GROUP1 has 4 observations and 2 variables.
```

```
NOTE: The data set WORK.GROUP2 has 3 observations and 2 variables.
NOTE: The data set WORK.GROUP3 has 3 observations and 2 variables.
NOTE: There were 10 observations read from the data set WORK.HAVE.
NOTE: DATA statement used (Total process time):
      real time           0.00 seconds
      cpu time            0.01 seconds
```

How does it work?

The first DECLARE HASH statement creates H. A brief digression here. It was not yet explained that H is a "special" type of variable that holds value of pointer to hash object. In earlier releases of SAS, it could be seen in the LOG if you have used the _NEW_ statement to instantiate a hash object[5].

the log - H is a variable in PDV

```
35   %put &sysvlong.;
9.01.01M3P020206
36   data _null_;
37     x=42;
38     declare hash H;
39     H = _new_ hash();
40     y="don't panic";
41     put _all_;
42   run;

x=42 H=5.6817E-299 y=don't panic _ERROR_=0 _N_=1
NOTE: DATA statement used (Total process time):
      real time           0.00 seconds
      user cpu time       0.00 seconds
      system cpu time     0.00 seconds
```

The HoH uses this in very clever way, it uses the fact that H can be assigned to data portion of a hash table. The second DECLARE HASH statement does exactly that. A HoH hash table is created, the X variable is the key and in the data portion that "special type" variable H is placed. The iterator is to make iteration over HoH easy.

The DO-UNTIL loops over input data set. Inside the body of the loop three things happen. The first step checks if the X value, which was just read into the PDV, exists in the HoH. If not, this means that the current value of X is a new, not yet met, value for which a brand new instance of hash H has to be created. The _NEW_ statement does the thing, a new pointer is written into H. Then, labels and "defining" methods create the structure of that instance of hash object. Lastly the .REPLACE() method adds new entry with current X value and associated pointer to the HoH. The second step calls the .FIND() method on HoH to pull the right hash object pointer (associated with X value in PDV) into H variable in the PDV, so that the third step could be executed. The third step calls the .ADD() method on H associated with current value of X. There is no need for rc= because the multidata:"Y" was used during declaration and, by the way, the ordered:"D" solved ordering by Y. When the input data set is read top to bottom the memory layout can be imagine as in the figure below. "Invisible" hash table pointers in the PDV have been marked with black background, and values of those pointers are represented as #N. All hash tables are populated with data. Instances that do not have pointer value located in the PDV have grayish headers. Iterator points outside the HoH hash table. Everything is ready to write out content of hash tables to output data sets.

[5]Thank you, Paul Dorfman, for showing me this trick!

Program Data Vector

_E_[num,8]	X[num,8]	Y[num,8]	Z[char,8]	_N_	_ERROR_	H[hash,?]	HoH[hash,?]
1	3	19	J	1	0	#3	#0

HoH [#0 hash]

X[num,8]	X[num,8]	H[hash,?]
1	1	#1
2	2	#2
3	3	#3

◀ I

H [#1 hash]

Y [num,8]	Y [num,8]	Z [char,8]
17	17	H
15	15	F
12	12	C
10	10	A

H [#2 hash]

Y [num,8]	Y [num,8]	Z [char,8]
18	18	I
13	13	D
11	11	B

H [#3 hash]

Y [num,8]	Y [num,8]	Z [char,8]
19	19	J
16	16	G
14	14	E

The loop at the bottom of the DATA step uses the iterator `I` to traverse HoH and extract pointers to all other hash tables residing in memory and awaiting for output. When `.NEXT()` method executes the `H` variable is updated with pointers values one-by-one. When a pointer is located in the PDV, the `.OUTPUT()` method can be executed and data from a proper hash table are written out to a data set.

CONCLUSION

Hash tables were discussed within SAS conferences for more than 44 years! Yes, you have read right, 44 years. The first article describing the technique I was able to refer to is [**W. D. Phillips, R. C. Race, SUGI 1982**], there were 3 more I could find in the '80 (all listed in the bibliography section), but the real bloom of the discussion started with Paul Dorfman's publication [**P. M. Dorfman, SUGI 2000**]. Since then, after hash objects became a SAS thing (see [**J. Secosky, WUSS 2002**]), the subject exploded with hundreds of publications, literally. As of October 12, 2026 (including this article) I have managed to catalog two books describing the subject *and* 471 article spreading across 327 titles and 245 authors (see Appendix B - saslexicon query). And I am willing to bet $5 I have missed a bunch! The subject of working, using, and writing better performing programs with hash tables has grown and is still expanding (articles and books mentioned exceeded over 7000 pages). I hope this paper at least in some extent helped you to understand hash tables better.

The End

REFERENCES

[P. Dorfmand & D. Henderson 2018] Paul M. Dorfman, Donald J. Henderson
"Data Management Solutions Using SAS Hash Table Operations: A Business Intelligence Case Study",
SAS Institute Press, June 2018, ISBN: 1629601438

[M. M. Burlew 2012] Michele M. Burlew
"SAS Hash Object Programming Made Easy",
SAS Institute Press, September 2012, ISBN: 9781607648017

[B. Jabłoński, PharmaSUG 2025] Bartosz Jabłoński, *"SET Statement Considered Harmful"*,
Pharmaceutical Software Users Group 2025 Proceedings,
https://pharmasug.org/proceedings/2025/AP/PharmaSUG-2025-AP-121.pdf

[J. Loren, R. A. DeVenezia, SGF 2011] Judy Loren, Richard A. DeVenezia, *"Building Provider Panels: An Application for the Hash of Hashes"*,
SAS Global Forum 2011 Proceedings,
`https://support.sas.com/resources/papers/proceedings11/255-2011.pdf`

[P. M. Dorfman, K. Vyverman, SUGI 2005] Paul M. Dorfman, Koen Vyverman, *"Data Step Hash Objects as Programming Tools"*,
SAS Users Group International 2005 Proceedings,
`https://support.sas.com/resources/papers/proceedings/proceedings/sugi30/236-30.pdf`

[J. Secosky, WUSS 2002] Jason Secosky, *"The DATA Step in Version 9: What's New?"*,
Western Users of SAS Software 2002 Proceedings,
`https://www.wuss.org/proceedings/2002/wuss2002.121.tt.pdf`

[P. M. Dorfman, SUGI 2000] Paul M. Dorfman, *"Private Detectives in a Data Warehouse: Key-Indexing, Bitmapping, and Hashing"*,
SAS Users Group International 2000 Proceedings,
`https://support.sas.com/resources/papers/proceedings/proceedings/sugi25/25/dw/25p129.pdf`

[J. E. Johnstone, C. K. Ray, SUGI 1989] James E. Johnstone, Craig K. Ray, *"A Survey of Table Lookup Techniques for the SAS System"*,
SAS Users Group International 1989 Proceedings,
`https://support.sas.com/resources/papers/proceedings-archive/SUGI89/Sugi-89-05%20Johnstone%20Ray.pdf`

[C. K. Ray, SUGI 1988] Craig K. Ray, *"Implementation of a Hashing Routine in SAS Software"*,
SAS Users Group International 1988 Proceedings,
`https://support.sas.com/resources/papers/proceedings-archive/SUGI88/Sugi-13-235%20Ray.pdf`

[A. Utlsch, SEUGI 1987] A. Utlsch, *"From Information Retrieval to Knowledge Based System"*,
SAS European Users Group International 1987 Proceedings,
`https://support.sas.com/resources/papers/proceedings-archive/SEUGI1987/From%20Information%20Retrieval%20to%20Knowledge%20Based%20System.pdf`

[W. D. Phillips, R. C. Race, SUGI 1982] William D. Phillips, Roger C. Race, *"Teaching Programming and Documentation Techniques"*,
SAS Users Group International 1982 Proceedings,
`https://support.sas.com/resources/papers/proceedings-archive/SUGI82/Sugi-82-28%20Phillips%20Race.pdf`

ACKNOWLEDGMENTS

The author would like to acknowledge Louise Hadden for her help in proofreading and polishing linguistic aspects of the final article.

CONTACT INFORMATION

Your comments and questions are valued and encouraged. Contact the author at one of the following e-mail addresses:

`yabwon`✉`gmail.com` or `bartosz.jablonski`✉`pw.edu.pl`

or via the following LinkedIn profile: `www.linkedin.com/in/yabwon`
or at the `communities.sas.com` by mentioning `@yabwon`.

SAS and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration.
Other brand and product names are trademarks of their respective companies.

Appendix A - code coloring guide

The best experience for reading this article is in color and the following convention is used:

- The code snippets use the following coloring convention:

code: is surrounded by a black frame

```
In general we use black ink for the code but:
- code of interest is in orange ink so that it can be highlighted,
- to distinguish code snippets for easier reading dark blue is used,
- and comments pertaining to code are in a bluish ink for easier reading.
```

- The LOG uses the following coloring convention:

the log - is surrounded by a blueish frame

```
The source code and general log text are blueish.
Log NOTEs are green.
Log WARNINGs are violet.
Log ERRORs are red.
Log text generated by the user is purple.
```

Appendix B - saslexicon query

To list articles tagged as hash table related or having titles containing hash keyword visit

https://saslexicon.org/quicktitlessearch.html

and search for "hash table|iter" phrase among titles and keywords. Or use the following link:
saslexicon.org/quicktitlessearch.html?fields=title%2Ckeywords&q=hash+table%7Citer
As of October 12th the query lists 470 positions.

INDEX

concept
ASSOCIATIVE ARRAY, 4
DOT NOTATION, 4
HASH ITERATOR, 5, 30
HASH OBJECT, 1
HASH OF HASHES, 49
HASH TABLE, 1, 4
METHOD, 4
SAS ARRAY, 2, 3
SAS TEMPORARY ARRAY, 3

function
.ADD(), 7–9, 12, 15, 22, 23, 25, 30, 51
.CHECK(), 4, 26, 30
.CLEAR(), 27, 28
.DEFINEDATA(), 6, 8, 10, 17, 19, 20, 22, 24
.DEFINEDONE(), 7, 10, 24
.DEFINEKEY(), 6, 8, 10, 17, 19, 20, 24
.DELETE(), 27, 28
.DO_OVER(), 36–38
.FIND(), 9, 22, 25, 26, 35, 36, 45, 51
.FIND_NEXT(), 35, 36
.FIND_PREV(), 36
.FIRST(), 31, 32, 34
.HAS_NEXT(), 36
.HAS_PREV(), 36
.LAST(), 31
.NEXT(), 31–34, 45, 49, 52
.NUM_ITEMS, 4, 27
.OUTPUT(), 16–18, 20, 22, 45, 52
.PREV(), 31, 49
.REF(), 30
.REMOVE(), 27, 29, 30
.REMOVEDUP(), 29, 30
.REPLACE(), 30, 45, 51
.REPLACEDUP(), 30
.RESET_DUP(), 38
.SETCUR(), 34
CALL EXECUTE, 48
CALL MISSING, 9, 14, 22
DIM, 4
SUBSTR, 7, 8
VNAME, 24

I/O(input/output), 45–47

option
_METHOD, 45
LABEL, 11
POINT=, 3
WHERE=, 12

procedure
SQL, 45

special variable
ERROR, 6
_N_, 6, 42

statement
NEW, 10, 31, 51
TEMPORARY, 3
ARRAY, 7
ASSIGNMENT, 8
DATA, 17
DECLARE HASH, 6, 10, 51
DO-LOOP, 7, 36
DO-UNTIL, 35, 45, 49, 51
DO-WHILE, 34, 36, 45, 49
IF-THEN, 9, 14
LENGTH, 7, 13, 14, 47
SET, 3, 13, 14, 40, 44, 47
STOP, 14, 40